AN ENGINEERING & RESEARCH AUTOBIOGRAPHY

The Story of Building PlantDx

How a single, stubborn idea — never let a model that can't diagnose a disease describe one — grew into a deterministic, knowledge-grounded pipeline for fine-tuning vision–language models on plant disease, and what its author learned at every turn.

Disease Knowledge Base · Deterministic Ontology · Closed Vocabulary · 1,966-caption corpus · QLoRA · Qwen2.5-VL · MLX · Two-stage Evaluation

Reconstructed **only** from repository evidence — 16 commits, 12 tags, the full spec set, configs, generated reports, and evaluation metrics. Nothing invented; gaps are marked **"cannot be reconstructed from repository evidence."**

Compiled from the state at commit **9f08d99** on **main** · Author of record: **iAakash1** · sole GitHub contributor.

Before you begin — how this story was reconstructed

This document is written as a story, but it is not a work of imagination. Every event, decision, number, and bug in it is drawn from artifacts that live inside the PlantDx repository: its git history, its frozen specifications, its configuration files, its generated reports, and its evaluation metrics. The narrative voice is a *device*; the facts underneath it are not.

So that you can trust it, here are the rules I held myself to while writing:

THE FOUR RULES OF THIS RECONSTRUCTION

1. **Evidence or silence.** If a claim is in this document, it is traceable to a file, a commit, a config, or a report. Most chapters end with an *Evidence* box naming the source.
2. **No invented events.** There are no fabricated meetings, conversations, dates, or durations. Where the story would need one and the repository does not record it, it says *"This cannot be reconstructed from repository evidence."*
3. **WHY over WHAT.** The repository already documents *what* exists. This document exists to explain *why* — the problem that forced each decision, the alternatives weighed, and the trade-off accepted.
4. **The first person is the project's, not a fiction.** The "I / we" voice narrates the reasoning that the specs, ADRs, and commit messages record. It never asserts an inner state the evidence doesn't support.

A note on authorship. The git history shows one human author of record (`iAakash1`) and commits co-authored with an AI assistant. When this story says "we," it means the engineering effort as the repository records it — the decisions, not a roster. When it says "I learned," it is naming a lesson the *codebase itself* teaches, visible in how a later commit corrects an earlier one.

A note on scope. You will meet exactly the models, datasets, and tools that actually appear in the repository — no more. Where you might expect others (a CNN bake-off, certain VLMs named in passing by readers), the story says plainly that the repository contains no such evidence, rather than filling the gap.

WHAT THIS STORY DELIBERATELY DOES NOT CLAIM TO KNOW

The author's personal biography and motivation; the specific papers that first sparked the idea; any pre-repository experiments (for example, a ResNet/EfficientNet/MobileNet/CLIP classification bake-off, or a cloud/A100 training attempt) — **none of these exist as artifacts in the repository**, so the story reconstructs the *reasoning* the specs preserve and marks the rest as unknowable from the evidence. Likewise, GitHub carries **zero issues, pull requests, and releases** for this project, so no collaborative-discussion thread could be reconstructed; the "conversation" of this project happened in commits, specs, and code.

How to read the callout boxes

Box	What it holds
Story	A narrative aside — the human texture of a moment in the build.
Decision	A major choice, dissected: *Problem → Assumption → What we tried → Why it failed → Alternatives → Trade-offs → Decision → Implementation → Consequences → Lesson.*
Postmortem	A real bug, told like an incident report: symptom, hypotheses, cause, fix, prevention.
Evolution of my thinking	What I believed before this chapter, what changed my mind, and the principle I carried forward.
Lesson	The transferable takeaway, stated plainly.
Evidence	The files, commits, and reports that ground the surrounding claims.
Cannot be reconstructed	An honest gap — something the story will not invent.

Table of Contents

Part I · Before Day One — the idea 5

Why this problem, why grounded captions, why no model in the loop 5

Part II · The first prototype was a repository, not a program 10

Scaffolding first — and the first lesson: cut the abstraction 10

Part III · Meeting the data 12

The PlantVillage surprise, the audit engine, and normalization 12

Part IV · Writing down the truth — the Disease Knowledge Base 15

Why cite APS, UC IPM, IFAS, CABI; why keep disagreements honest 15

Part V · The realization the DKB alone wasn't enough — the ontology 18

Duplicated logic, and the intermediate representation that killed it 18

Part VI · Bounding the words — vocabulary & lexicon 22

The closed-vocabulary bet, and two bugs the real data caught 22

Part VII · Teaching the machine to speak — concepts, templates, corpus 25

Why captions were terrible first, and the three grammar bugs that fixed them 25

Part VIII · Training 30

Why Qwen, why MLX, why QLoRA, why local — and the response pool 30

Part IX · Evaluation — the chapter that deserved a book 33

Two stages, official metrics, and 7% → 94% . . . 33

Part X · When mango broke everything . . . 37

A random-chance score, three hidden literals, and crop-agnosticism . . . 37

Part XI · The demo . . . 40

A confidence of one-in-a-billion, and how it became honest . . . 40

Part XII · Making it public . . . 42

Re-render the figures; never recompute a metric . . . 42

Part XIII · Where PlantDx stands today — and what it still cannot do . . . 44

Reflections · Things we got wrong & things we deleted forever . . . 47

Reflections · Assumptions that changed & decisions we'd never repeat . . . 49

Reflections · The hardest problem & the most satisfying breakthrough . . . 51

Reflections · If we rebuilt it today, and if we had six more months . . . 53

The evolution of the engineer . . . 55

Appendix · Evidence index & provenance . . . 57

PART I

Before Day One — the idea

Every project has a sentence it is secretly about. PlantDx has one, and everything else in this document is a consequence of it: *a model that cannot reliably diagnose a plant disease must never be trusted to describe one*. If you understand why that sentence is load-bearing — not a slogan, but a constraint that shaped the database, the ontology, the vocabulary, the templates, the training, and the evaluation — you already understand PlantDx better than any file listing could teach you.

But to feel why that sentence mattered, you have to start before the first commit, with the problem it was answering.

Why this problem at all

PlantDx is a framework for building instruction-tuning datasets for *agricultural* vision–language models — specifically, models that look at a single photographed leaf and produce a grounded, clinical description of what disease it shows. The domain is narrow on purpose: ten tomato conditions from PlantVillage and eight mango conditions from MangoLeafBD, eighteen classes in total, all of them *leaf* conditions.

Why agriculture, and why these two crops? The honest answer has two halves. One half is reconstructable and the other is not, and I will not blur them.

> **CANNOT BE RECONSTRUCTED**
>
> The repository does not record the author's personal reason for choosing agriculture, tomato, or mango, nor any private motivation, nor the specific papers that first inspired the project. Those belong to a person, not to a codebase, and inventing them would violate the first rule of this document. What *is* in the repository is the scientific framing of the problem and the datasets that were actually used — and that is what the rest of this chapter reconstructs.

The reconstructable half is this. The two datasets are *single-leaf images*: PlantVillage tomato leaflets on plain backgrounds, and MangoLeafBD single mango leaves. That physical fact — one leaf, plain background, no fruit, no tree, no field — turns out to be the second most important thing about the whole project, because it defines exactly what a truthful caption is *allowed* to say. A disease can be scientifically famous for the cankers it forms on twigs or the rot it causes in fruit; but if you are looking at one leaf, and you write "twig canker," you have written something false about *this picture*, no matter how true it is about the disease. Hold that thought; it becomes the observability constraint.

Why grounded captioning, and not a classifier

The obvious way to attack "what disease is this leaf?" is classification: feed the image to a network, read off a label. So the first real design question was whether PlantDx should be a classifier at all. It chose to be a *captioner* — a model that emits a grounded description from which a label can be extracted — and the reasoning, preserved in the methodology spec, is worth reconstructing carefully, because a lot of readers will assume a CNN was tried and rejected. It was not, and I will not pretend otherwise.

CANNOT BE RECONSTRUCTED

There is **no evidence in the repository of a CNN experiment** — no ResNet, EfficientNet, MobileNet, or CLIP-classification baseline, trained or benchmarked, anywhere in the code, configs, reports, or history. So this document does *not* claim "we tried a CNN and it lost." What it can do is explain, from the frozen methodology spec, why the project's *target* was a grounded caption rather than a bare label.

Decision · Captioning as the target, classification as a read-out

PROBLEM

The datasets give a label per image. What should the model be trained to *produce* — the label, or a description?

ORIGINAL ASSUMPTION

A label is the deliverable, so predict the label.

WHY THAT FELT WRONG

A bare classifier has nothing to *ground*. When it is right you cannot see why; when it is wrong you cannot see why; and — crucially for a research artifact whose whole pitch is traceability — a label carries no evidence. A grounded caption, by contrast, carries the very observations that justify the label ("concentric target rings with a yellow halo on older leaves"), and those observations are exactly what a knowledge base can license and a validator can check.

TRADE-OFF ACCEPTED

Captioning is harder to evaluate than classification (you need caption-quality metrics *and* a way to extract a label for accuracy). We accepted that cost — and, tellingly, the evaluation pipeline later pays it in full, computing both.

DECISION

Train the model to emit a grounded, knowledge-licensed caption. Derive the classification metric *downstream*, by extracting the disease name from the caption text.

CONSEQUENCE

This one choice shaped the entire language layer (concepts → templates → corpus) and the evaluation's `extract_disease_id` step. It is why PlantDx has a "corpus of captions" at all.

LESSON

Choose the output that carries its own justification. A label is an answer; a grounded caption is an answer *with its reasons attached* — and reasons are what you can audit, cite, and trust.

The founding constraint: no model in the caption loop

Now the sentence from the top of the chapter. Once you decide the training target is a *caption*, you face the question that quietly dooms most caption datasets: *where do the training captions come from?* The path of least resistance in 2020s ML is to ask a big model to write them — prompt a VLM on each image, or prompt an LLM from the label, and harvest the text. PlantDx refused this outright, and made the refusal the axis the whole architecture turns on.

The methodology spec states the reason with unusual bluntness, and it is an empirical one, not an aesthetic one: general-purpose open-weight VLMs are *unreliable* at crop-disease description, so using their output as training targets would encode their errors into the student. It even names the trap precisely — the **circularity** of model-distilled supervision: *using a model that is weak at a task to synthesize the labels that will teach that task.*

> **STORY · THE NUMBER THAT JUSTIFIED EVERYTHING**
>
> There is a moment, much later in this story, that retroactively vindicates this whole stance — and it is worth previewing here because it is the emotional core of the project. When the base Qwen2.5-VL model was finally measured zero-shot on the held-out test sets, it scored **7.3% accuracy on tomato** and **17.5% on mango** — at or near random for the class counts. Read the per-disease breakdown and the base model's "strategy" is almost poignant: it calls nearly everything *healthy*, or it hallucinates. On tomato it got 81% of the healthy leaves right and essentially 0% of every actual disease; on the 269 yellow-leaf-curl images it hallucinated a wrong description 71% of the time. *This* is the model some pipelines would have asked to write their training captions. The founding constraint was not paranoia. It was correct.

Decision · Ban every model — LLM and VLM — from the caption path

PROBLEM

Training captions have to come from somewhere. The cheap source is a generative model.

ORIGINAL ASSUMPTION

A capable VLM/LLM can write "good enough" captions we then filter.

WHY IT FAILS

Three ways, all structural: *circularity* (a weak model's output becomes the labels that teach the task); *hallucination* (endemic, and worst exactly in specialized domains like agriculture); and *observability leakage* (a model freely asserts fruit, twig, or whole-tree features that cannot be in a single-leaf photo). Post-hoc filtering catches known failure modes; it cannot catch the ones nobody thought to test.

ALTERNATIVES CONSIDERED

(A) VLM-on-image distillation — rejected: inherits the model's errors and can "read" the class from spurious cues. (B) LLM-from-label generation — rejected: unverifiable priors, no grounding. (C) Hand-written templates only — insufficient alone (no controlled diversity, no evidence links). (D) *Generation-by-knowledge* — chosen.

TRADE-OFF ACCEPTED

We give up open-ended lexical spontaneity. In exchange we get supervision whose correctness is a property of a *curated, cited knowledge base*, and whose every token is auditable. The spontaneity is bought back later, deliberately, by controlled diversity mechanisms.

DECISION

Nothing in the caption-generation path may call an LLM or a VLM, or read image pixels. A caption is a pure function of `(disease_label, knowledge base, seed)`.

CONSEQUENCE

This is the seed of the DKB, the ontology, the closed vocabulary, and the deterministic generator — every one of them exists to make this ban *true by construction* rather than by good intentions.

It is worth being precise about what the ban buys, because it is the project's single research contribution, and the spec argues it as a table. Reconstructed, the contrast is:

Property	Knowledge-grounded (PlantDx)	LLM/VLM-distilled
Factual grounding	Every claim traces to an authoritative KB field	Model priors; unverifiable
Hallucination	Structurally prevented (closed vocabulary + validators)	Endemic; filtered post hoc
Circularity	Impossible — no model output enters supervision	A weak model's outputs become the labels
Observability control	Enforced (single-leaf constraint in the KB)	May assert fruit/tree/field features
Reproducibility	Bit-for-bit, fully seeded and provenanced	Sampling- and version-dependent
Auditability	Total — every term licensed and traceable	Opaque; needs external verification

THE EVOLUTION OF MY THINKING · BEFORE THE FIRST LINE OF CODE

What I believed before: that the bottleneck in building an agricultural VLM was *data volume* — that more captions, from whatever source, would win.

What changed my mind: the recognition that a caption dataset inherits the *epistemics* of whatever produced it. If a model wrote it, the dataset can only ever be as trustworthy as that model — and in this domain, that is not trustworthy at all.

The principle I carried forward: *provenance beats volume.* A smaller corpus where every sentence traces to a cited source is worth more than a large corpus you have to take on faith. Every later decision — determinism, content hashes, fail-closed validators — is this principle wearing different clothes.

EVIDENCE

`caption_framework/07_ieee_methodology_section.md` §III-B, §III-K (the no-model argument and the contrast table); `caption_framework/README.md` (the seven design invariants, esp. #1 label-only grounding and #6 severity honesty); `knowledge_base/dkb.json → metadata.observability_constraint` (single-leaf rule); the base-model failure numbers from `reports/qwen25vl_{tomato,mango}_qlora/evaluation/per_disease.csv` (base rows). The absence of any CNN/CLIP baseline was verified by full-repository search.

PART II

The first prototype was a repository, not a program

The first commit of PlantDx — tagged `v0.1-scaffold`, 162 files, ~8,900 lines — contains almost no working logic. That was the point. Before a single caption was generated, the project built the *shape* of a mature research system: a `src/` package, a config system, a CLI with every future subcommand stubbed, a test scaffold, and typed interfaces for every pipeline component that would ever exist. Each stub was an honest promise: a function that raised `NotImplementedError` pointing at the milestone that would fill it in.

Why build the skeleton before the muscles

There is a real argument on both sides here, and the repository comes down firmly on one. Building the scaffold first is a bet that the *architecture* is the hard part, not the code — that if you nail the boundaries between components early, the implementations can land one at a time without the whole thing turning to mud. The bet paid off in a very specific way: for the rest of the project's life, every milestone had a place to live before it was written. When the ontology compiler arrived, its package already existed as a stub; when evaluation arrived, its CLI verb was already reserved.

> **STORY · THE HONESTY OF A GOOD STUB**
>
> A stubbed function that raises `NotImplementedError("Milestone 4: ...")` is a strange kind of documentation: it tells you, at the exact call site, what is real and what is a promise. Reading the scaffold commit is like walking through the framing of a house — no walls yet, but every room is already the right size, and you can see where the plumbing will go. This is why, chapters from now, you will keep meeting the phrase "the stub was already there."

And immediately, the first correction: cut the abstraction

What makes the scaffold milestone genuinely instructive is not that it was built — it is what happened *next*. A review pass went back over the fresh scaffold and started *deleting*. This became a pattern that recurs through the entire history, so it is worth watching it happen the first time.

BEFORE REVIEW

`core/interfaces.py` — a layer of generic `Protocol` classes duplicating the concrete base classes.

`SeedDeriver` — a class wrapping what were really three pure functions.

`dataset/converters/` — seven one-line files, one per converter.

`dataset/schema.py` — a name that collided with `config/schema.py`.

AFTER REVIEW

`interfaces.py` **deleted** — nothing consumed it; the concrete ABCs sufficed.

`seeding.py` — three pure functions (`image_seed` , `caption_seed` , `attempt_seed`).

One `dataset/converters.py` — base ABC + 5 converters + registry.

Renamed to `dataset/serialization.py` — the collision gone.

LESSON · THE CHEAPEST ABSTRACTION IS THE ONE YOU DELETE BEFORE IT GROWS ROOTS

The codebase's own history states the principle: it is easier and cheaper to consolidate over-abstraction *after* the fact than to prevent every instance of it up front — but prefer not needing to. A `Protocol` layer with zero consumers is not "extensibility," it is speculative weight. The reflex this instilled — *if nothing calls it, it should not exist* — is the same reflex that later kept the ontology package to eight single-purpose files and refused a heavyweight RDF reasoner.

THE EVOLUTION OF MY THINKING · THE SCAFFOLD

Before: that a "serious" codebase signals its seriousness through abstraction — interfaces, factories, layers.

What changed my mind: deleting `interfaces.py` and watching nothing break. The seriousness was in the *boundaries* and the *tests*, not in the indirection.

Carried forward: *simple, explicit, less clever*. No dependency injection, no visitor patterns, no generic frameworks. A plain function or a small dataclass until proven otherwise.

EVIDENCE

Commit `12efddd` (tag `v0.1-scaffold` , 162 files); `CHANGELOG.md` "post-review simplification (no methodology change)" — the deletions of `core/interfaces.py` , the `SeedDeriver` →functions refactor, the converters consolidation, and the `schema.py` → `serialization.py` rename.

PART III

Meeting the data

Every dataset lies a little about itself until you count it. The first thing PlantDx did with real files was not train, or generate, or even look at a picture — it *audited*. And the audit immediately found something that rewrote the plan.

The PlantVillage surprise

The working assumption was that the tomato data was a tidy 10-class tomato subset of PlantVillage. The audit engine — which walks every image, reads its metadata, and counts classes — reported otherwise: the folder held the **complete, multi-crop PlantVillage dataset**, roughly 54,000 images across **38 classes** spanning tomato, potato, corn, grape, apple, and more. Ten of those classes were the tomato conditions the project wanted; the other twenty-eight were noise for this purpose.

> **STORY · THE DISCOVERY THAT CREATED A WHOLE ENGINE**
>
> This is the moment the audit engine justified its own existence on its first real run. Had the project skipped straight to training, it would have fed a 38-class haystack to a pipeline expecting a 10-class needle. Instead, a read-only counting pass caught the mismatch before it could become a training bug — and the fix it demanded (extract only the relevant classes, canonically) is exactly why the *next* component, the normalization engine, exists. One dataset surprise spawned two engines' worth of design.

Why the audit never decodes a pixel

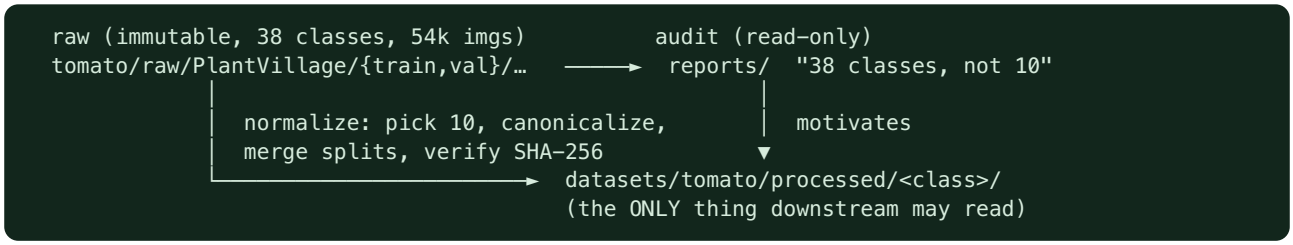
A subtle, deliberate choice hides in the audit engine: to inspect 58,305 images (54,305 tomato + 4,000 mango) it reads *metadata only* — dimensions and format via Pillow, never a full decode. Two reasons. First, speed: a full decode of 58k images is minutes you pay every run. Second, and more in character for this project, *robustness*: a single corrupt image should never crash an inventory. The engine is built so one bad file is a logged finding, not a stack trace. On the real data it found **zero** corrupt images and 21 exact-duplicate groups per dataset.

> **LESSON**
>
> An inventory tool's job is to survive the data, not to trust it. Inspect the least you need (metadata, not pixels), and make a bad input a *finding*, not a failure.

Why normalization became necessary

The audit told the truth; normalization acted on it. The normalization engine extracts *only* the relevant classes from each raw dataset, renames them to canonical slugs (`Tomato___Early_blight` → `early_blight`), merges any train/val split into unified class directories while recording each image's original split in a manifest, verifies every copied file's SHA-256 against its source, and writes the

canonical tree `datasets/<crop>/processed/<class>/` that every downstream stage is *required* to consume — never `raw/`.



Two design rules here are load-bearing for the rest of the project. First, **raw is immutable** — the 28 non-tomato classes are *ignored, not deleted*; they stay untouched in `raw/` and are recorded by name in `class_mapping.json:ignored_folders`. Second, **consumers read the normalized layout, never raw** — the same discipline the ontology later applies to the DKB ("derive from the compiled artifact, not the source prose"). This is the first appearance of a rule that becomes a theme: *each stage consumes the previous stage's canonical output, not its raw input.*

A bug we chose to leave — and documented instead

POSTMORTEM · THE MANIFEST THAT REPORTS ZERO

Symptom. Run `plantdx normalize --dataset mango` a second time and it cheerfully reports *"0 images across 1 crop(s)."* The manifest, class map, and report all say zero; the checksum is `e3b0c442…` — the famous SHA-256 of the empty string.

First hypothesis (wrong). Data loss — did the re-run wipe `processed/`? No. Every previously normalized file is still on disk, correctly named. Nothing was deleted.

Actual cause. The idempotency that makes re-runs fast is exactly what fools the report. On the second run, every source file's hash matches a file already in place, so each is classified `"duplicate"` and — correctly — not re-copied. But the loop that builds the manifest only appends *freshly placed* files. So the manifest describes "what this run did" (nothing) rather than "what `processed/` currently contains" (everything).

Fix (identified, deliberately deferred). On a `"duplicate"`, still append the existing file's record instead of skipping it — so the manifest always reflects the full current state. Left unimplemented because it is a genuine control-flow change to a behavior-frozen engine, and it surfaced during a pass explicitly scoped to zero behavior change.

Prevention / honesty. Rather than a silent gotcha, it lives in `docs/KNOWN_ISSUES.md` with the symptom, the cause, why it is safe, and the one-line workaround (`rm -rf datasets/<crop>` before re-running).

LESSON · A KNOWN BUG BEATS A HIDDEN ONE

Not every bug should be fixed the instant it is found. A cosmetic reporting defect in a frozen component, fixed hastily, risks a real regression; the disciplined move is to *write it down completely* — symptom, cause, blast radius, fix location — and defer it on purpose. The danger is never the bug you have documented.

THE EVOLUTION OF MY THINKING · THE DATA STAGE

Before: that datasets arrive as described, and preprocessing is a chore.

What changed my mind: discovering 38 classes where 10 were promised. The dataset's self-description was wrong, and only *counting* revealed it.

Carried forward: *audit before you architect*. Never build on an assumed shape of the data; measure it first, and make the measurement a reproducible artifact, not a one-off glance.

EVIDENCE

Tag `v0.3-normalization` (commit `3e73d0f`); `CHANGELOG.md` M2 (Audit) and M2.1 (Normalization) entries; `docs/KNOWN_ISSUES.md` §1 (the idempotent-manifest postmortem, verbatim cause and deferral rationale); audit findings (0 corrupt of 58,305; 38 raw tomato classes) from the M2 changelog and dataset metadata.

PART IV

Writing down the truth — the Disease Knowledge Base

If no model is allowed to supply disease facts, then *something* has to. That something is the Disease Knowledge Base — `knowledge_base/dkb.json` — 18 conditions, 46 fields each, every scientific claim tied to a citation. It is hand-curated, it is declared FINAL, and it is the single source of truth from which the ontology, the vocabulary, every template slot, and every caption ultimately descend. This chapter is about why it had to be built by hand, and why by hand *with citations*.

Why not just ask an LLM to assemble the facts?

It is tempting to think the no-model rule applies only to *caption writing*, and that an LLM could at least gather the disease knowledge. But the same circularity argument bites one level deeper. If an LLM assembles the "facts," then the knowledge base — the very thing meant to be the trustworthy floor beneath everything — is only as reliable as that LLM's priors in a domain where those priors are weakest. The knowledge base is where the buck stops; it cannot itself be a model's guess. So it was assembled the slow way: from authoritative plant-pathology sources, with each disease's claims referenced back to them.

> **STORY · WHY A HUMAN-READABLE JSON FILE IS A SUPERPOWER**
>
> The DKB is one file a domain expert can open and check. "Is this symptom right for early blight?" is answerable by a plant pathologist reading one record — not by archaeology across a codebase. Centralizing scientific truth in a single reviewable, version-controlled, human-readable file is what turns "are the captions correct?" from a research risk into a code-review question.

Why these sources, and why cite at all

The DKB carries a `reference_registry` of **21 cited sources**, and every disease's claims point into it. The registry is a roll-call of the field's standard authorities — university extension services and professional-society compendia, backed by peer-reviewed taxonomy papers:

Source family	Examples in the registry
University extension / IPM	`UCIPM-Tomato`, `UCIPM-Mites`, `UFIFAS-PP351`, `UFIFAS-HS1369`, `UMN-LeafMold`, `CTAHR-PD46`
Professional society (APS)	`APS-BSGuide`, `APS-CompTomato`
CABI / global compendia	`CABI`, `CABI-Deporaus`
Peer-reviewed taxonomy	`Videira2017`, `Constantin2016`, `Timilsina2019`, `Sossah2024`, `Chomnunti2011`, `Ploetz2003`, `Agrios2005`
Dataset provenance	`MangoLeafBD`, `NHB-Mango`, `PHA-GallMidge`, `Arauz2000`

The citation is not decoration. Later, the ontology enforces a rule (`V-ONT-4` / `V-ONT-7`) that *every* assertion edge must resolve to an `Evidence` node in this registry, or the build fails. That is the mechanism that turns the paper-sentence "every caption claim traces to a cited source" from an aspiration into something a validator can prove. The citations in the DKB are the roots of that traceability tree.

The 46 fields, and the two that make honesty possible

Each disease record spans identity, causal agent, ecology, symptomatology, controlled vocabularies, management, and provenance. Most of the 46 fields are exactly what you would expect of a plant-pathology record. Two are unusual, and they exist to make the project's core honesty invariants *enforceable*:

- `forbidden_symptoms_not_leaf_observable` — the disease's real symptoms that are *not* visible on a single leaf (fruit lesions, twig cankers, whole-tree decline, yield loss). They are recorded precisely so the pipeline knows what to *forbid*. This is the observability constraint given a home in the data.
- The split `severity.{mild,moderate,severe}` vs. extent vocabulary — the seed of severity honesty. The DKB knows what "severe" *typically* looks like, but the datasets never label per-image severity, so the pipeline may describe visible *extent* ("scattered," "numerous") but never assert a per-image *stage* ("severe").

> **STORY · NEGATIVE KNOWLEDGE IS KNOWLEDGE**
>
> Most datasets only tell a model what *is* true. The DKB deliberately encodes what must *never* be said: `forbidden_symptoms_not_leaf_observable`, `forbidden_terms`, `forbidden_adjectives`. This "negative knowledge" is what lets a validator, much later, reject a caption for a symptom that is scientifically real but photographically impossible. The DKB is as careful about the boundary of the visible as it is about the visible itself.

Keeping scientific disagreements honest

A lesser knowledge base would flatten uncertainty into a single confident answer. The DKB does the opposite: it records **four documented taxonomic disagreements** rather than pretending they are

settled — and this honesty flows all the way downstream, because the ontology later represents a contested cause as *parallel* `caused_by` edges, each with its own evidence, rather than a single edge.

Class	The honest complication (recorded, not hidden)
Tomato bacterial spot	A four-taxon species complex (*X. euvesicatoria* pv. *euvesicatoria* & pv. *perforans*, *X. hortorum* pv. *gardneri*, *X. vesicatoria*) — not one pathogen.
Tomato leaf mold	Accepted name unsettled: *Fulvia fulva* (Videira 2017), with synonyms *Passalora fulva* / *Cladosporium fulvum*.
Mango "Bacterial Canker"	The MangoLeafBD label = the disease usually called bacterial black spot (*Xanthomonas citri* pv. *mangiferaeindicae*).
Mango "Die Back"	As a *leaf* class it captures the leaf-level scorch of a twig/branch/tree-decline syndrome (*Lasiodiplodia theobromae*).

Remember that last row. The die-back class is flagged, right here in the knowledge base, as a leaf-level shadow of a whole-plant syndrome — meaning its leaf appearance is generic browning and scorch that overlaps with other conditions. When, much later, the fine-tuned model struggles with exactly one mango class, it will be this one, and the DKB will have predicted the difficulty in advance.

THE EVOLUTION OF MY THINKING · THE KNOWLEDGE BASE

Before: that a "knowledge base" is a convenience — a place to stash facts so code can look them up.

What changed my mind: realizing the DKB is not a lookup table but the project's *epistemic floor*. Everything the system is ever allowed to claim is bounded by this one file, so its citations and its honesty about uncertainty are not niceties — they are the guarantee.

Carried forward: *centralize truth, and cite it*. One reviewable source of scientific fact, with provenance on every claim, beats knowledge scattered across code where no expert can check it.

EVIDENCE

`knowledge_base/dkb.json` — `metadata.reference_registry` (21 sources), `metadata.documented_taxonomic_disagreements` (4 entries, verbatim), `metadata.observability_constraint`, `metadata.pest_vs_pathogen_note`, and the 18 disease records (10 tomato + 8 mango) with agent categories. Field taxonomy and the FINAL/single-source-of-truth status from `caption_framework/README.md` and `caption_framework/00_methodology_overview.md` §1.

PART V

The realization the DKB alone wasn't enough — the ontology

Here is the redesign the whole project pivots on. We had a beautiful, cited knowledge base. We thought it was enough. It was not — and understanding *why* is the difference between reading PlantDx as a pile of components and reading it as an argument.

The problem: the same logic, drifting in three places

The DKB stores its truth as *prose fields*. That is perfect for a human reviewer and terrible for a consumer. Because the moment more than one downstream stage needs to answer a question like *"is this symptom observable on a leaf?"* or *"may we claim this with confidence?"*, each stage has to re-parse the same prose and re-implement the same rule. And rules that live in multiple places drift apart. The vocabulary builder's idea of "observable" and the caption validator's idea of "observable" would, over time, quietly diverge — and nothing would tell you.

> **STORY · THE DUPLICATION YOU CAN FEEL BEFORE YOU CAN SEE**
>
> The spec names the smell exactly: without an intermediate layer, "every downstream stage would re-parse DKB prose, re-encode the observability rule, and re-derive confidence — duplicating logic, drifting over time, and defeating traceability." You do not need the drift to have happened yet to know it will. One change to the knowledge base would require chasing the observability rule through every consumer that had re-implemented it. That is the maintenance nightmare the ontology was invented to prevent.

What we tried, and the alternatives we weighed

The decision was not made lightly; the ontology design document lays out three options and argues the trade-off explicitly.

Decision · Insert a typed knowledge graph between the DKB and everything else

PROBLEM
Downstream stages keep re-parsing DKB prose and re-implementing the observability and severity rules, risking silent drift and making the grounding claim informal.

ALTERNATIVE A — CONSUME THE DKB PROSE DIRECTLY (THE STATUS QUO)
Simplest short-term. Rejected: every stage re-implements parsing + observability + confidence; the grounding claim stays informal; adding crops becomes risky. It does not scale.

ALTERNATIVE B — A FULL OWL/RDF ONTOLOGY WITH A REASONER
Maximally standard and interoperable. Rejected as the primary representation: a heavyweight dependency, non-trivial nondeterminism across reasoner versions, and an implementation cost at odds with the project's "simplest correct implementation" ethos. Kept only as an *optional future export target*.

CHOSEN — A CONSTRAINED TYPED PROPERTY GRAPH
A minimal, JSON-expressible schema (concept types, relation types, domain/range, inheritance, constraints, provenance) with a deterministic builder. "90% of the semantic-web benefit at 10% of the cost, and trivially deterministic." The design doc calls this "the load-bearing decision of the whole design."

TRADE-OFF ACCEPTED
We forgo standard RDF tooling and formal reasoning power we do not need, in exchange for determinism, a light dependency footprint, and full control over the build.

CONSEQUENCE
"Is the caption grounded?" becomes a *graph-reachability query*: `Condition –has_symptom→ Symptom –supported_by→ Evidence`. Observability becomes a structural property (a symptom's `appears_on` points at a leaf region or it does not). The rule now has exactly one implementation.

The four-way split that keeps a caption scientifically sane

The ontology's concept hierarchy has one distinction that is worth singling out, because it is doing quiet, essential scientific work. Every condition is one of four kinds — and the split is not bureaucratic taxonomy, it is what stops a caption from committing a category error:

Condition subtype	Examples	What the split prevents
`Disease`	early blight, late blight, leaf mold…	—
`PestDamage`	spider mites, cutting weevil, gall midge	Calling feeding/mechanical stippling a "lesion" or "infection."
`SurfaceColonization`	sooty mould (a saprophyte on honeydew)	Calling a superficial coating "necrosis" — the leaf tissue is not even infected.
`HealthyState`	tomato/mango healthy	Ever attaching a `caused_by` edge to a healthy leaf.

The DKB's own `pest_vs_pathogen_note` demands this: for pests and for sooty mould, use "pest/feeding/surface vocabulary, never infection/lesion vocabulary." The ontology encodes that demand as *types*, so the constraint is checked structurally rather than remembered by a template author.

Fail-closed, and the hash that became a covenant

The ontology compiler is *fail-closed*: eleven blocking validators (`V-ONT-1..11`) run at build time — schema conformance, domain/range, cardinality, ten named forbidden-relation rules (e.g. a `HealthyState` may never have a `caused_by` edge; a `typical_at_severity` edge must always be `image_licensed=false`), evidence closure, and an observability recomputation that *re-derives* each symptom's observability from its anatomy edges and checks it matches. Any single violation aborts the build with **no artifact written**. There is no such thing, on disk, as an invalid ontology.

And the output carries a `content_hash` computed *only* from semantic content — never timestamps, paths, or machine identity. For the real 18-condition DKB it compiles to:

```
ontology content_hash =
sha256:25ae0f6d9692a6d00a8968dc916a3665001bba29dd45616afe5e9b3c49bf2ca4
   574 concepts · 1,289 edges · 18 conditions · 40 concept types · 15 relation types
```

STORY · A HASH YOU ARE NOT ALLOWED TO BREAK BY ACCIDENT

That string is checked into a test as a golden value. It is treated as sacred: any change to it is "a semantic event requiring a reviewed, deliberate version bump — never a silent side effect." Later, an entire CI-cleanup pass touching 120 lint findings had one non-negotiable acceptance criterion — *the ontology hash must be unchanged* — and it was verified unchanged at every step. The hash turned reproducibility from a hope into a tripwire.

LESSON · PUT THE RULE IN EXACTLY ONE PLACE, THEN MAKE VIOLATING IT IMPOSSIBLE

An intermediate representation earns its keep when it collapses a rule that was going to be re-implemented N times into *one* implementation — and then a fail-closed validator makes "forgot to apply the rule" unrepresentable. Determinism plus a content hash makes "changed the meaning by accident" unrepresentable too. The ontology is not a database convenience; it is the instrument that makes the project's claims checkable.

THE EVOLUTION OF MY THINKING · THE ONTOLOGY

Before: that the DKB, being cited and structured, was already the foundation — that consumers could just read its fields.

What changed my mind: seeing that "read its fields" secretly means "re-implement its rules," once per consumer, forever. The foundation was truth, but it was truth in a form that invited drift.

Carried forward: *compile prose into structure once*. A typed intermediate layer with fail-closed validation is not over-engineering when it is the thing that makes a scientific claim mechanically verifiable — and "simplest correct" meant a property graph, not an OWL reasoner.

EVIDENCE

Tag `v0.4-ontology` (commit `2125fc9`); `ontology_design/01_architecture.md` §1.1–1.6 (the why, the problem table, and the A/B/chosen trade-off analysis); the four-way `Condition` split from `ontology_design/03_concept_hierarchy.md`; `knowledge_base/dkb.json → metadata.pest_vs_pathogen_note`; the golden hash and 574/1,289 statistics from the ontology changelog entry and `tests/unit/ontology/test_domain_determinism.py`.

PART VI

Bounding the words — vocabulary & symptom lexicon

A caption is made of words, and words are where hallucination sneaks back in. You can ban models from the loop and still lose if a template is free to reach for any adjective that "sounds right." So the next stage built a fence around the language itself: a closed, ontology-derived controlled vocabulary, from which — and *only* from which — a caption may draw its colors, shapes, textures, and extents.

Why a closed vocabulary is a bet worth making

An open vocabulary means a template (or, worse, a future model) could introduce a term that is plausible but unlicensed — a color no source ever attributed to this disease. A closed vocabulary makes that impossible by construction: every descriptor is a node the ontology put there, traceable to the DKB field that licensed it. There is a second, subtler payoff that the ontology design calls the **vocabulary-saturation argument**: the set of ways to describe a leaf lesion's color and shape in plant pathology is *small and it saturates fast*. The 500th disease you add introduces essentially no new color or shape nodes. This is why the graph stays small — 18 conditions give 574 concepts, and even a hypothetical 500-disease, 12-crop expansion is projected to only ~7–8k nodes. A closed vocabulary is not a limitation you tolerate; it is what makes the whole thing scale.

The quiet decision to skip "component A" — for now

The original caption-framework spec had the vocabulary builder (component B) and symptom lexicon (component C) consume a *caption-concept model* (component A) that did not yet exist. Rather than block on A, the milestone re-founded B and C directly on the compiled ontology — the same pattern the ontology used against the DKB. Everything B and C needed (closed-vocabulary value nodes, symptom nodes with their DKB phrases, evidence-linked relations) was already in the ontology's A-Box. So they read the ontology, never the DKB, never the unbuilt A.

> **LESSON · DON'T BLOCK ON A COMPONENT YOU CAN ROUTE AROUND**
>
> When a downstream stage depends on an upstream one that isn't built, ask whether the *substrate* it really needs already exists somewhere else. B and C nominally needed "A"; what they actually needed was licensed value-nodes and symptom phrases — and those lived in the ontology. Re-founding on the real dependency, not the nominal one, unblocked two components at once.

Two bugs the synthetic tests missed and the real data caught

This milestone is the first vivid instance of a theme that recurs to the end of the project: **the real 18-condition DKB catches bugs that hand-written test fixtures do not.** Two of them are worth telling in full, because both were found by *inspecting real output*, not by a green test suite.

POSTMORTEM · "RAISED GALL," EMITTED TWICE

Symptom. The symptom lexicon for a few mango conditions contained duplicate realizations — "raised gall" appearing twice for the same symptom.

Cause. A handful of DKB conditions legitimately use the *same word* on two different axes: `mango_gall_midge` has `shape: raised` *and* `texture: raised`; `mango_powdery_mildew` has `shape: coalescing` *and* `extent: coalescing`. Realizing every modifier edge independently produced the same phrase from two different nodes.

Fix. Sort each symptom's candidate modifier edges by a fixed axis priority (color → shape → texture → extent, via `MODIFIER_RELATIONS`), and skip any edge whose resulting phrase was already emitted for that symptom. The dropped axis is never lost — it still appears as its own vocabulary entry; it just does not *also* produce a duplicate lexicon phrase.

Prevention. The synthetic fixture was then *extended* to reproduce this exact collision — a regression test named for it — so the fixture finally covered what only real data had exposed.

POSTMORTEM · THE AGENT THAT CALLED ITSELF `AGENT:ALTERNARIA_SOLANI`

Symptom. Agent-name vocabulary items read like `"agent:alternaria_solani"` — a raw node id — instead of `"Alternaria solani"`.

Cause. Every node type stores its display name under `canonical_label` — *except* causal agents, which store it under `scientific_name`. The first builder used `get("canonical_label", node.id)` uniformly, so for every agent it silently fell back to the id.

Fix. One shared helper, `graph_queries.node_label(node)`, checks the node type and reads `scientific_name` for agents, `canonical_label` for everything else. The builder, the lexicon, and the validator's independent re-derivation all route through it — one source of truth for "what is this node called."

Why the tests missed it. The synthetic fixture happened to exercise agents correctly in isolation; only reading the *actual* `vocabulary.json` revealed the label content was wrong.

The output of this stage, on the real DKB, is 268 vocabulary items and 983 bounded lexicon realizations, byte-identical across runs, pinned by:

```
vocabulary content_hash =
sha256:99d5833cf6405eea8edf01b227411c5db572393e6dfe27323ce6519b29249628
   268 vocabulary items · 983 lexicon items (259 base + 724 single-modifier) · fail-closed
V-VOC-1..9
```

Note the word *bounded*. The lexicon deliberately refuses a Cartesian product: a modifiable symptom gets one verbatim base phrase plus *one linear* single-modifier phrase per attached quality value —

"brown lesion," not "circular brown coarse extensive lesion." Multi-modifier stacking was consciously deferred. Linear, not combinatorial, is what keeps the surface auditable.

THE EVOLUTION OF MY THINKING · VOCABULARY

Before: that grammar and validators would be enough to keep captions honest, so vocabulary could be generous.

What changed my mind: realizing that an open vocabulary re-opens the exact hallucination door the no-model rule closed — a plausible-but-unlicensed term is a hallucination even if a human typed the template. And that only *real* data surfaces the messy collisions (a word that is both a shape and a texture) that a tidy fixture never will.

Carried forward: *close the vocabulary, and verify against reality.* Bound the linguistic surface at the source, keep realizations linear rather than combinatorial, and never trust a green fixture suite over a look at the actual output.

EVIDENCE

Tag `v0.5-vocabulary` (commit `7b45bb2`); `CHANGELOG.md` vocabulary entry (the cross-axis collision, verbatim); the saturation argument from `ontology_design/06_statistics.md` and `CLAUDE.md` §10.5; the agent-label fix in `src/plantdx/vocabulary/domain/graph_queries.py`; the golden hash and 268/983 counts from `tests/unit/vocabulary/test_vocabulary_domain_determinism.py` and `docs/VOCABULARY.md`.

PART VII

Teaching the machine to speak — concepts, templates, corpus

This is the chapter the whole first half was building toward: the moment PlantDx finally produces English. And it is the chapter with the most honest lesson in the project, because the first captions it produced were, frankly, bad — and the bad captions are where the real engineering happened. A pipeline that emits polished sentences from a synthetic fixture teaches you nothing. A pipeline that emits *"a coating are visible"* from the real knowledge base teaches you exactly what is broken.

Three components, one job

The language layer is three collaborating pieces, deliberately separated so that *syntax* and *semantics* never contaminate each other:

```
DKB + Ontology + Vocabulary
      │
      ▼
(A) CONCEPT MODEL   per disease: required / optional / forbidden concepts,
    concepts/        canonical order, info budget, register, sign type,
                     controlled realizations + evidence, never_appear set
      │
      ├──────────────► (E) TEMPLATE ENGINE   33 authored templates, 8 families,
      │                    templates/          structured lit/slot/opt/list segments;
      │                                         carry SYNTAX only, zero disease facts
      ▼                          │
(planner) fill slots with seed-chosen realizations ──► SentencePlan
      ▼
(generator) realize to English + deterministic surface repair
      ▼
(validator) 12 independent checks — never trusts the generator
      ▼
(builder) enumerate bounded, de-duplicated, validated per-disease corpus
      ▼
captions.{json,jsonl,csv}  +  5 export formats
```

Why the concept model reads the DKB — and not just the ontology

Here is a genuinely surprising design turn, and it is worth dwelling on because it corrects an assumption the earlier chapters might have planted. You would expect component A to be a pure *view* over the ontology — the spec even framed it that way. It is not, and the reason is instructive.

Decision · A reads the DKB, cross-linked to the ontology — not the ontology alone

PROBLEM

The concept model needs fine-grained, per-symptom concept distinctions (chlorosis vs. necrosis vs. leaf-curling vs. lesion-distribution) plus each concept's controlled realizations.

ORIGINAL ASSUMPTION

A is a deterministic projection of the ontology — read the graph, done.

WHY THAT FAILED

The ontology *collapses* all symptoms into a single `Symptom` type and does not keep the DKB's dedicated `chlorosis` / `necrosis` / `leaf_curling` / `lesion_distribution` fields as distinct concepts. Projecting the ontology alone would lose exactly the distinctions the concept model is *for*.

DECISION

A reads the DKB (its designed input) for the concept structure, cross-linked to the ontology and vocabulary for evidence, confidence, sign-types, and controlled realizations. It is a *join*, not a pure view.

CONSEQUENCE

Two honesty rules fall out immediately and permanently: `lesion_size` is **omitted** (there is no controlled DKB size vocabulary, so inventing one would violate no-invention), and `severity_stage` and `management` are **always forbidden** concepts (severity + observability honesty).

LESSON · RESPECT WHAT EACH REPRESENTATION IS GOOD AT

The ontology is the right tool for licensing and observability queries; it is the *wrong* tool for fine symptom distinctions it deliberately collapsed. "Everything downstream reads the compiled artifact" is a great rule until the compiled artifact dropped the detail you need — then you read the source, honestly, and say so.

Why templates use structured segments, not inline strings

The 33 templates are the caption *syntax*, and they carry zero domain knowledge — no disease name, no symptom, no adjective ever lives in a template. All of that enters through slots. But *how* a template represents its optional parts turned out to matter enormously.

THE SPEC'S INLINE-STRING STYLE

A template as one pattern string with optional fragments spliced in, then repaired with regex when a slot is empty. Fragile: delete an optional slot and you must fix the surrounding articles, commas, and spaces by pattern-matching — the classic source of "a , visible" glitches.

PLANTDX'S STRUCTURED SEGMENTS

A template as a list of typed segments — `lit` / `slot` / `opt` / `list`, each with its own `glue` and `suffix`. Deleting an optional slot is *grammatical by construction*: the segment simply isn't emitted, and no regex repair is needed. Structure, not string surgery.

LESSON · MAKE THE INVALID STATE UNREPRESENTABLE

The inline-string approach makes broken grammar a thing you must remember to repair. The structured-segment approach makes it a thing that cannot occur. Whenever you catch yourself writing repair code for a bad state, ask whether a different representation makes that state impossible in the first place.

The validator that refuses to trust the generator

The corpus builder's validator is a battery of **twelve independent checks** (`V-CAP-1..12`) that never assume the generator did its job. It re-checks forbidden symptoms, observability, closed-vocabulary membership, required content, pest/pathogen register, cross-disease term leakage, a severity-stage guard, internal contradictions, grammar, and duplication. If a candidate fails, it is dropped and *recorded*; if a disease yields zero valid captions, the build hard-errors — correctly blaming the knowledge base or its derivation rather than silently shipping a thin class.

RC1: the hardening pass where real English fought back

Before the corpus was frozen, it went through a quality-only hardening pass (RC1) run *against the real DKB*. Three real grammar bugs surfaced that no synthetic fixture had caught — each one a small, deterministic fix, each one a lesson about the gap between "passes tests" and "reads like English."

POSTMORTEM · THE THREE GRAMMAR BUGS OF RC1

Bug 1 — the clause used as a noun. DKB phrases like "begin on oldest leaves" or "forms a film" were dropped into a `{primary}` slot, producing "*begin on oldest leaves are visible*." *Fix:* a deterministic noun-phrase filter rejects primary-sign realizations that are really verb clauses (later extended to reject trailing finite verbs like "young leaves distort").

Bug 2 — number disagreement. "*a coating are visible*," "*spider mites is present*." Singular filler, plural verb, and vice-versa. *Fix:* the few number-fragile templates were made number-*neutral* — "can be seen," "Signs of X are present" — so agreement can't break.

Bug 3 — negation leak. DKB phrases like "none characteristic" leaked in as if they were positive realizations. *Fix:* a first-word negation filter drops them at the source.

Why it matters. Every one of these was invisible to the fixtures and obvious the instant the generator ran on the real 18-condition knowledge base. This is the same lesson vocabulary taught, now in the language layer: *run against reality*.

RC1 did more than fix grammar; it improved the corpus measurably. The healthy class — previously starved of concepts — was decomposed into multiple atomic, evidence-backed observations from the DKB's healthy fields, taking healthy captions from **12 → 32**. Bounded per-template variants drawing on each disease's multiple DKB phrasings took the whole corpus from **1,070 → 1,966** captions, with the per-disease minimum rising **6 → 15**. And `severity_stage` tokens were filtered at the source, dropping the `V-CAP-11` severity-rejection count from **17 → 0** — the validators kept as defense-in-depth, but nothing left for them to reject.

What a grounded caption actually looks like

Abstract enough. Here are real captions, straight from the frozen corpus — each one a pure function of label, knowledge base, and seed; each one traceable to the DKB fields that licensed it:

Class	A real caption from the corpus
tomato early blight	*"Diagnosis: early blight. Findings: target/bull's-eye concentric rings with yellow halo on older leaves; lower/older leaves first."*
tomato spider mites	*"Diagnosis: two-spotted spider mite. Findings: fine stippling and webbing (and mites, if visible)."*
mango sooty mould	*"Diagnosis: sooty mould. Findings: black-to-dark-brown superficial sooty/velvety coating on the leaf surface (typically adaxial); patchy to continuous."*
tomato healthy	*"Tomato leaf, no pathology. Normal turgid, slightly pubescent surface; margins intact."*
tomato yellow leaf curl virus	*"Diagnosis: tomato yellow leaf curl virus. Findings: small, upward-cupped, yellowing leaflets on new growth."*

Look at what these *don't* say. The spider-mite caption uses "stippling" and "webbing" — pest language — never "lesion" or "infection." The sooty-mould caption says "superficial coating," because the leaf tissue is not infected. Neither asserts a severity stage. Every restraint you can see is a rule from an earlier chapter, now visible in the output. The corpus is pinned by:

```
corpus content_hash =
sha256:1225eb8605edb71839f89618bfdcd4a1abba414869f64c1fdc5951ee865ba089
   1,966 captions · per-disease minimum 15 · fail-closed V-CAP-1..12 · byte-identical
rebuilds
```

STORY · THE CORPUS IS IMAGE-FREE ON PURPOSE

Note what the corpus is *not*: it is not tied to any image. It is a disease-level library — 1,966 captions describing the 18 conditions, generated with no picture in sight. That is deliberate. The images enter only at the *next* stage, where each caption becomes a *response* paired with a real leaf photo. Keeping the corpus image-free is what lets it be a pure, hashable function of the knowledge base — and it is why every caption above could be regenerated, bit-for-bit, on any machine, forever.

THE EVOLUTION OF MY THINKING · THE CAPTION LAYER

Before: that once the knowledge, ontology, and vocabulary were correct, producing English would be a formality — plug slots into templates.

What changed my mind: "*a coating are visible.*" The knowledge was right, the vocabulary was licensed, and the sentence was still wrong. Grounding guarantees *truth*; it does not guarantee *grammar*, and the two need separate machinery.

Carried forward: *separate syntax from semantics, and harden against real inputs before freezing.* Structured segments make bad grammar unrepresentable; running on the real DKB — not fixtures — is the only thing that finds the sentences that are true but unspeakable.

EVIDENCE

Tags `v0.6-generation` / `v0.6-corpus-freeze` (commit `05a98e1`); `CHANGELOG.md` M3 + RC1 entries (the three grammar fixes, healthy 12→32, corpus 1,070→1,966, severity rejections 17→0, verbatim); real captions from `artifacts/corpus/captions.jsonl`; the A-reads-DKB rationale and `lesion_size` / `severity_stage` omissions from `CLAUDE.md` §5 (M3) and `docs/CONCEPTS.md`; the golden corpus hash from `tests/unit/corpus/test_corpus_determinism.py`.

PART VIII

Training

Everything so far produced *supervision*: a knowledge base, a graph, a vocabulary, a corpus of 1,966 grounded captions. Now, finally, a model gets to learn from it. But even here — the one place a model is *supposed* to be — the founding discipline holds: the training pipeline reads image *paths and folder labels only*. It never opens a pixel, and it never asks a model to write anything. The corpus is the answer key; the images are just where the questions point.

The models that actually appear — and the ones that don't

Because readers reasonably ask "which models did you try?", here is the exact, evidence-bounded truth. There is one model with trained adapters and measured results; three more are *declared targets* with scaffolded converters but no trained weights; five are *export formats* (not models); and several models that outside readers sometimes mention appear **nowhere** in the repository.

Status in the repository	What it means	Which ones
Trained & evaluated	Real QLoRA adapters exist; base-vs-fine-tuned metrics are reported for both crops.	**Qwen2.5-VL-7B-Instruct-4bit**
Declared target · scaffolded	Named in the DKB's `target_models` and wired as converter classes — but not trained.	Qwen3-VL-8B, InternVL3-8B, Gemma-3-12B
Export format (not a model)	Serialization shapes the corpus reshapes into.	`generic`, `llava`, `paligemma`, `blip2`, `messages`
Absent entirely	Zero occurrences anywhere in code, config, or history.	Molmo, Pixtral, Florence

CANNOT BE RECONSTRUCTED

Any story about training or rejecting Molmo, Pixtral, or Florence — a full-repository search finds **no trace** of them. They are not part of PlantDx. Similarly, the three declared targets (Qwen3-VL, InternVL3, Gemma-3) have converter scaffolding but **no trained adapters and no metrics**; the story does not pretend they were fine-tuned.

Why Qwen, why MLX, why QLoRA, why local

Four choices, one coherent motivation: *a self-contained artifact that runs and reproduces on the hardware at hand* — an Apple M4 Pro with 24 GB of unified memory.

Decision · Local QLoRA on Qwen2.5-VL-4bit via mlx-vlm

PROBLEM
Fine-tune a 7B vision–language model within 24 GB of unified memory, reproducibly, with no external service.

WHY NOT FULL FINE-TUNING
A 7B model's full weights and optimizer state do not fit 24 GB. Parameter-efficient tuning is not a preference here; it is a constraint.

WHY QLORA "BY CONSTRUCTION"
Selecting the **4-bit** MLX base model and attaching **LoRA** adapters *is* QLoRA — quantized base + low-rank adapters. It fell out of the model choice. Only **40,370,176** adapter parameters train (rank 16), a fraction of a percent of 7B, at ~8–14 GB peak.

WHY MLX / APPLE SILICON, NOT CLOUD/A100/HF TRAINER
Local MLX means no API keys, no external service, no data egress — which serves the project's reproducibility ethos directly. The pipeline *orchestrates* `mlx_vlm.lora` (a tested trainer) rather than hand-rolling an MLX loop, isolating the trainer's shifting schema behind one command builder.

TRADE-OFF ACCEPTED
A single-machine, single-vendor toolchain (MLX) and a small trainable capacity, in exchange for a fully local, reproducible, self-contained training story.

CONSEQUENCE
Both crops train and serve on one laptop; inference runs ~0.8–1.7 s/image at ~5.8 GB steady. The demo, later, launches with no cloud dependency at all.

CANNOT BE RECONSTRUCTED
The repository does not contain a rejected cloud/A100 run or a Hugging Face `Trainer` / `PEFT` attempt to compare against. The MLX/local choice is recorded as a *positive selection* aligned with reproducibility, not as the winner of a documented head-to-head. This story reflects that honestly.

The response pool: where the corpus meets the images

The training data builder is the first place the image-free corpus and the normalized images are joined. It cross-joins the frozen caption corpus — used *verbatim* as the **response pool** — with the normalized tomato (and later mango) images, emits `mlx-vlm` JSONL rows `{"image","question","answer"}`, and writes a manifest. Two invariants make the join trustworthy:

- **Image-grouped, disease-stratified splits.** Splits are grouped by image so no caption-level leakage can occur, and stratified by disease so every class is represented in every split. Deterministic SHA-256 fanout picks the assignments.
- **Paths and labels only.** The builder reads an image's *path* and its *folder label* — never its pixels, never a model. Invariant #1, still holding, at the last stage before the GPU.

Configuration is layered in three composable YAMLs — a *run* × a *model* × a *LoRA method* — folded into one typed, fail-closed `TrainingConfig`. LoRA / QLoRA / DoRA are selectable by config, and a backend guard *rejects* DoRA (unsupported by mlx-vlm 0.6.x) with a clear message rather than silently

running LoRA instead. The hyperparameters, for the record: learning rate 1e-4 (cosine, warmup 100), effective batch 8 (batch 1 × grad-accum 8), 1 epoch, grad-clip 1.0, LoRA rank 16 / α 32 / dropout 0.05, 4-bit base, image resize 448, seq 2048, seed 20260711.

STORY · THE MILESTONE THAT SHIPPED A WORKFLOW, THEN RAN IT

The training commit (`v0.8`) is careful to say it built the *workflow* and did not launch training inside that commit — `plantdx train --dry-run` previews everything (iters, effective batch, time/memory/disk estimates, LR schedule) and only the un-dry run launches `mlx-vlm` . The proof that training was actually run afterward is not in that commit's diff; it is in what exists *downstream*: real QLoRA adapters and a full base-vs-fine-tuned evaluation for both tomato and mango. The pipeline and the act of running it are two different things, and the repository keeps them honestly separate.

LESSON · ORCHESTRATE THE TESTED TOOL; ISOLATE ITS CHURN

The trainer with the shifting schema (`mlx-vlm`) is wrapped behind one command builder, so its churn touches exactly one function. Building on a tested trainer instead of a hand-rolled loop, and quarantining the part most likely to change, is why a trainer-format change never rippled through the pipeline.

THE EVOLUTION OF MY THINKING · TRAINING

Before: that "training" was the main event and the data plumbing was preamble.

What changed my mind: realizing that the value was almost entirely upstream — in the response pool and the leak-free splits — and that training was, by design, a thin, deterministic orchestration over a tested trainer. The interesting decisions were about *not* reading pixels and *not* leaking images across splits.

Carried forward: *the model is the easy part*. Guard the data boundaries (paths not pixels, image-grouped splits) and let a tested trainer do the gradient descent.

EVIDENCE

Tag `v0.8-tomato-training` (commit `8ad2726`) and `feat: add mango training pipeline` (`380b18e`); `CHANGELOG.md` M7 entry (paths-and-labels-only, DoRA guard, dry-run vs launch); hyperparameters from `configs/{train,models,lora}/` ; `knowledge_base/dkb.json → metadata.target_models` (the four declared models); `src/plantdx/dataset/converters.py` (`CONVERTER_REGISTRY`) and `src/plantdx/exporters/formats.py` (the five export formats). Molmo/Pixtral/Florence absence verified by full-repository search.

PART IX

Evaluation — the chapter that deserved a book

Everything before this chapter was a bet: that knowledge-grounded supervision would teach a model better than a model could teach itself. Evaluation is where the bet gets settled — and settling it honestly turned out to be as much engineering as the training itself. The result, previewed once and now told in full, is stark: on tomato, base accuracy **7.3% → fine-tuned 93.7%**; on mango, **17.5% → 82.0%**.

Why two stages, when one would be simpler

The evaluation is split into two stages — `inference` then `analyze` — and the reason is not architectural elegance. It is a dependency conflict that could not be wished away.

Decision · Two-stage evaluation, split by dependency stack

PROBLEM

Generating predictions needs `mlx-vlm`. Scoring them needs a heavy metrics stack — matplotlib, scikit-learn, scipy, nltk, `pycocoevalcap`, `rouge-score`, `bert-score` + torch. These two dependency sets do not want to live in the same environment.

DECISION

Stage 1 (lazy mlx-vlm) writes `predictions.jsonl`; stage 2 (lazy metrics stack) reads *only* that frozen artifact. The two are never installed together; a frozen JSONL file is the clean interface between them.

CONSEQUENCE

Each stage's dependencies are installable in isolation; the metrics stage never needs a GPU or the network (WordNet + the BERTScore backbone are cached once, up front). And because stage 2 consumes a frozen file, you can re-score without re-inferring.

Official metrics, never approximated

A recurring temptation in evaluation code is to "approximate" a metric — a quick BLEU, a rough ROUGE. PlantDx refused: BLEU-1..4 and CIDEr via `pycocoevalcap`, ROUGE-L via Google's `rouge-score`, METEOR via `nltk`+WordNet, BERTScore via `bert-score`, classification via scikit-learn. Disease-label extraction, hallucination detection, and clinical correctness are deterministic *lexicon matching* grounded in the frozen DKB and compiled vocabulary — **never an LLM judge.** (Of course: an LLM judge would smuggle a model back into the loop the entire project exists to keep out.)

POSTMORTEM · THE NUMBA IMPORT THAT WASN'T PLANTDX'S FAULT

Symptom. BERTScore-dependent tests failed with `ImportError: numpy.core.multiarray failed to import`.

Hypotheses. A PlantDx bug? A bad pin in `pyproject.toml`? Neither.

Cause. `bert-score`'s model-loading path transitively imports `librosa` → `numba`, and this shared environment's `numba==0.56.4` predates NumPy 2.x. A clean venv with no numba/librosa works fine — the fault is the environment, not the code.

Resolution (honest, not silent). Rather than patch around it, `text_metrics.py` detects this *specific* failure and reports the exact fix — `pip install -U "numba>=0.59" "llvmlite>=0.42"` — instead of a raw traceback; the affected tests skip cleanly with a clear reason, and were separately verified to pass with zero skips in a clean environment.

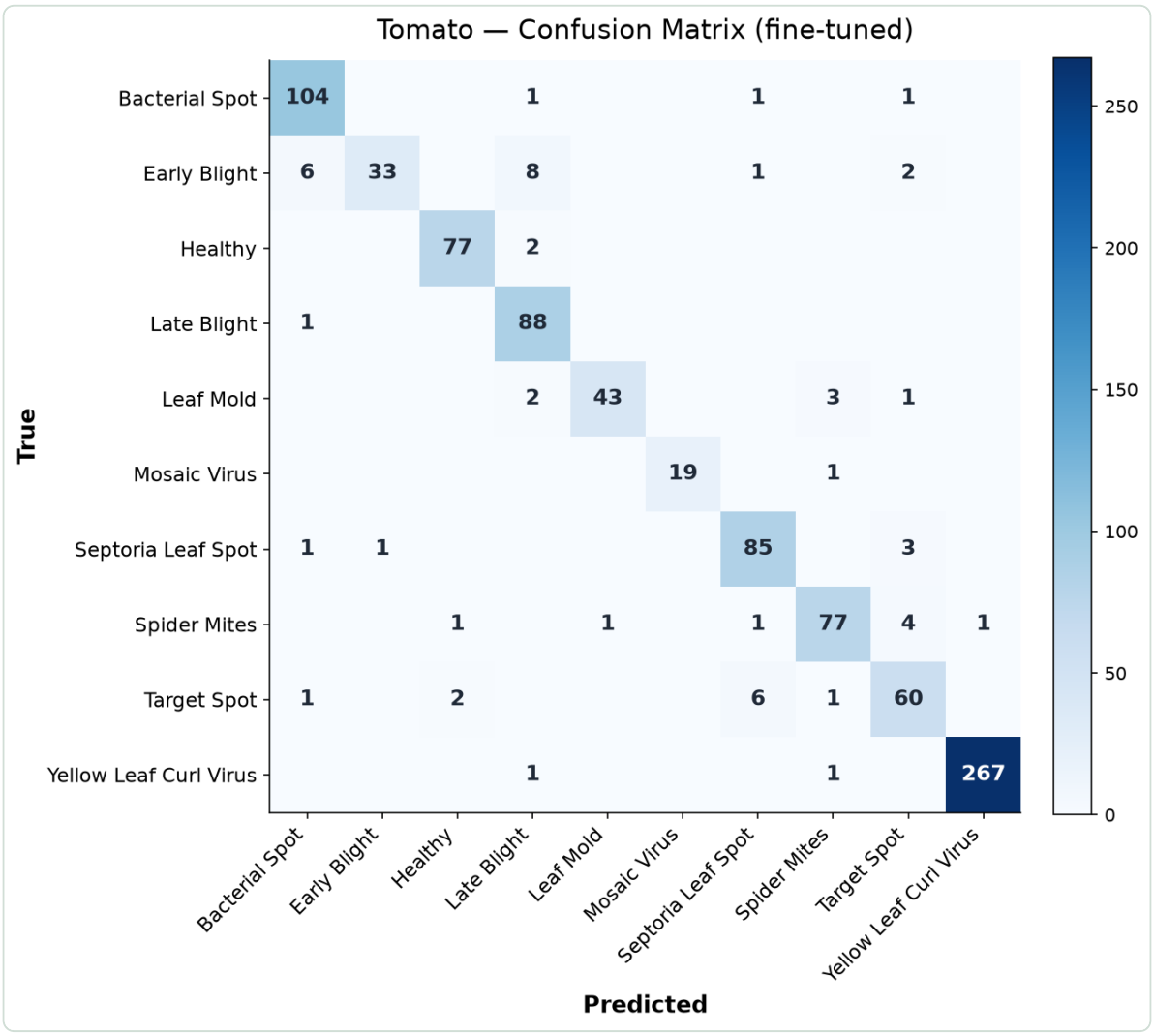
LESSON · DIAGNOSE THE ENVIRONMENT OUT LOUD

When a third-party dependency breaks for environmental reasons, the worst response is a silent workaround that hides a real, reproducible defect. The best is to detect the exact failure and hand the user the exact fix. An error message that teaches is worth more than a patch that conceals.

No leakage, by guard not by trust

Before any number is computed, a split-integrity guard reads the train split and the target split and hard-fails (`InvariantViolation`) on *any* image-path overlap. On the real frozen tomato dataset: **0 overlap**. The fairness of the comparison is not asserted; it is checked.

What the numbers say — and what the base model's failure reveals



Tomato — fine-tuned confusion matrix. A near-diagonal after fine-tuning; the residual confusion sits mostly in early blight (see below). Rendered from `reports/qwen25vl_tomato_qlora/evaluation/`.

Metric	Tomato base	Tomato fine-tuned	Mango base	Mango fine-tuned
Accuracy	7.3%	93.7%	17.5%	82.0%
Macro-F1	0.034	0.919	0.128	0.811
Weighted-F1	0.025	0.937	0.128	0.811
Test images	910	910	200	200

The per-disease breakdown tells a more human story than the aggregate. Read the *base* model's rows and you can see its coping strategy: it survives by calling things healthy or by hallucinating. On tomato it got **81%** of healthy leaves but essentially **0%** of every actual disease; on the 269 yellow-leaf-curl images it hallucinated a wrong description **71%** of the time. On mango it called **100%** of healthy leaves healthy — and **0%** of six of the seven diseases. This is the 7% and 17% laid bare: not a model that

knows a little, but a model that mostly does not know, defaulting to the safe word and the confident guess.

STORY · THE FOUNDING CONSTRAINT, VINDICATED IN A SPREADSHEET

Back in Part I, the ban on models-in-the-caption-loop rested on a claim: that a model this weak at diagnosis would be a catastrophic source of training captions. Here is that claim as data. A model that scores 7% and hallucinates on 7 in 10 yellow-leaf-curl images is precisely the wrong thing to ask for a caption — and precisely why PlantDx asked a cited knowledge base instead. The whole architecture was insurance against this spreadsheet, and the spreadsheet proves the premium was worth paying.

THE EVOLUTION OF MY THINKING · EVALUATION

Before: that evaluation was a final formality — run the metrics, print the table.

What changed my mind: discovering that *trustworthy* evaluation is itself a hard system — dependency-conflict management, leakage guards, official (not approximated) metrics, and a refusal to let an LLM judge back in. The honesty of the number depends on the engineering behind it.

Carried forward: *an evaluation you can't trust is worse than none.* Guard the splits, use reference implementations, isolate the dependency stacks, and never let the thing you're measuring grade its own homework.

EVIDENCE

Tag `v1.0-evaluation` (commit `2d11adf`); `CHANGELOG.md` M6 entry (two-stage split, official metrics, the numba/librosa bug verbatim, split-integrity guard, 0 overlap); metrics from `reports/qwen25vl_{tomato,mango}_qlora/evaluation/metrics.json` and `per_disease.csv` (base + fine-tuned rows); figure re-rendered by `scripts/render_readme_figures.py` into `docs/images/tomato_confusion_matrix.png`.

PART X

When mango broke everything

The most instructive bug in PlantDx did not come from a hard algorithm. It came from three little words — the string `"tomato"`, hardcoded in three places nobody thought of as "tomato-specific." The pipeline had been built, tested, and shipped tomato-first, and it worked beautifully. Then mango arrived, and mango scored like a coin flip.

The symptom: a random-chance model that had actually learned

The mango adapter had trained. It should have been good — the tomato adapter under the identical recipe reached 93.7%. But the evaluation reported mango predictions at roughly **chance** for eight classes, and the prediction file was full of wrong-looking labels: `unknown:*` extractions, or worse, a *tomato* disease id attached to a mango leaf. A model that had genuinely learned mango was being graded as if it had learned nothing.

POSTMORTEM · THREE HIDDEN `"TOMATO"` LITERALS

Symptom. Evaluating the real mango run produced near-random scores; predictions carried tomato disease ids or `unknown:*` instead of `mango_*`.

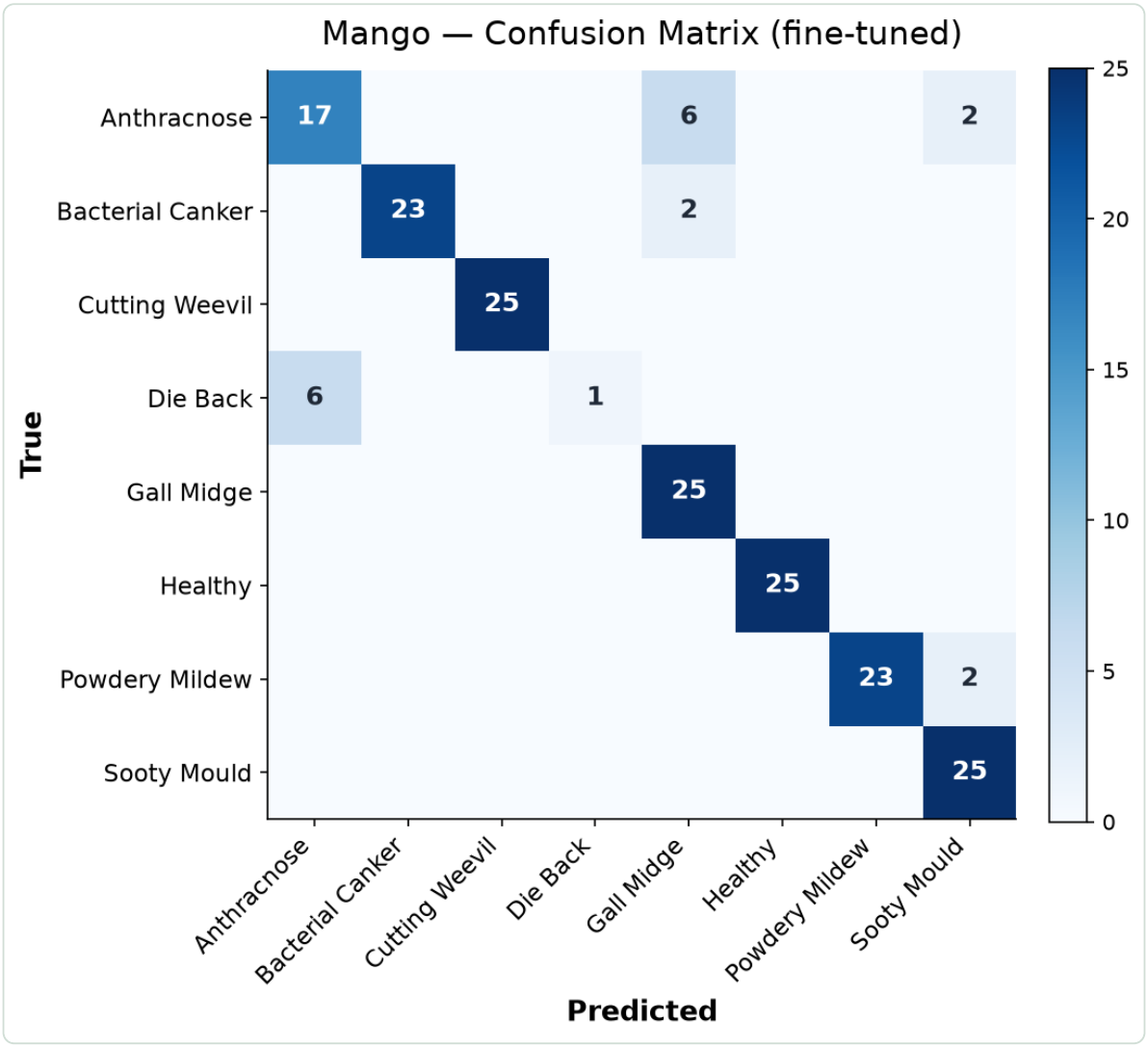
Hypotheses considered. (1) The mango model didn't learn — *ruled out*: the fine-tuned mango numbers, once fixed, are strong (82%). (2) The mango data was mis-split — *ruled out*: the split-integrity guard was clean. (3) The evaluation was interpreting mango through a tomato lens — *confirmed*.

Root cause — three hardcoded literals, each invisible until a second crop existed:

1. The inference runner resolved disease labels through `label_map("tomato")` — so a mango caption was matched against *tomato* disease names, extracting nonsense or a tomato-name collision.
2. The output directory defaulted to `reports/qwen25vl_tomato_qlora/evaluation` — a literal path, so mango results tried to land in the tomato folder.
3. Hallucinated-crop detection used a *static* crop list / tomato default, so "is this the wrong crop?" was computed against the wrong baseline.

The fix — make crop a derived value, never a constant. Crop is now *read from* `<dataset_dir>/manifest.json` — the same manifest the training pipeline writes. Stage 1 records the resolved crop into `metadata.json`; stage 2 reads it back to build the correct disease / hallucination / clinical lexicons. The output directory now derives from the adapter's own run name (`reports/<run_name>/evaluation`). Hallucinated-crop detection derives "other crops" from the DKB's own crop set minus the crop under test. No `--crop` flag needed anywhere.

Verification & prevention. Re-run against the real, previously-mislabeled mango run: predictions now carry correct `mango_*` ids; tomato evaluation re-verified unaffected. New regression tests target each symptom directly — output-dir derivation from the adapter path, fail-closed crop resolution, mango disease-id extraction, and per-crop hallucinated-crop true/false positives.



Mango — fine-tuned confusion matrix (after the crop-agnostic fix). Strong on the distinctive pest/mechanical classes; the one stubborn row is die back. Rendered from `reports/qwen25vl_mango_qlora/evaluation/`.

And notice, in that matrix, the row the knowledge base predicted back in Part IV. Mango **cutting weevil** and **gall midge** and **healthy** are 100% — the distinctive mechanical-damage and clean-leaf classes are easy. But **die back** lands at just **4% recall** (F1 0.077), even though when the model *does* say die back it is always right (precision 1.0). The model has learned to be cautious about a class whose leaf appearance — marginal browning, scorch, drying — is generic, exactly as the DKB warned when it flagged die back as "the leaf-level scorch of a twig/branch/tree-decline syndrome." The hardest disease to classify is the one the knowledge base already told us was ambiguous on a leaf.

LESSON · THE SECOND INSTANCE IS WHERE YOUR HIDDEN ASSUMPTIONS SURFACE

Everything works with one crop, one tenant, one currency — because the constant you hardcoded happens to be correct. The *second* instance is the audit. The fix here was not clever; it was a discipline: a domain value like "crop" must be *derived* from the data (the manifest), never embedded as a literal. After this, adding a third crop needs **zero** evaluation-code changes.

THE EVOLUTION OF MY THINKING · GENERALIZATION

Before: that "it works and the tests pass" meant the code was general. The tomato pipeline was clean, typed, well-tested — and silently tomato-shaped.

What changed my mind: watching a genuinely-learned mango model score like a coin flip because three strings said `"tomato"`. Correctness for one instance is not generality.

Carried forward: *derive domain values, don't embed them.* If a constant encodes "which instance are we," it is a latent bug waiting for instance number two. Read it from the data, and let the data be the single source of that truth — the same move the ontology made against the DKB, now applied to "crop."

EVIDENCE

Commit `673ed77` (tag `v0.2.0`, "generalize evaluation pipeline for multiple crops"); `CHANGELOG.md` crop-agnostic entry (the three literals and the manifest-derived fix, verbatim; "495 passed, 47 skipped"); the die-back difficulty and mango per-class numbers from `reports/qwen25vl_mango_qlora/evaluation/per_disease.csv`; the DKB's die-back caveat from `metadata.documented_taxonomic_disagreements`.

PART XI

The demo

A research artifact that only its author can run is a paper with extra steps. The Streamlit demo exists to let anyone upload a leaf and watch the fine-tuned model diagnose it — grounded caption, confidence, adapter proof, all in the browser. Building it surfaced one last bug that is a small masterpiece of "the obvious number is a lie."

A confidence of one in a billion

The demo needed a per-prediction confidence. The obvious source is the model's own log-probabilities. So the first version read them from the one-shot `generate()` call — and every prediction came back with a confidence around **1e-9**. One in a billion. For predictions that were, in fact, correct.

POSTMORTEM · WHY `GENERATE()` REPORTED 1E-9

Symptom. Confidence ≈ 1e-9 on every prediction, including confidently-correct ones — a meaningless, un-displayable number.

Cause. `generate()` exposes only the *last token's* logprobs over the *full vocabulary*. Normalizing one token's distribution over ~150k vocab entries yields a vanishingly small number that has nothing to do with the model's certainty about its *answer*.

Fix. Switch to `stream_generate` and compute confidence as the *mean probability of the model's own emitted tokens* — the tokens it actually chose, not the whole vocabulary. On correct predictions this yields a sane **~0.85–0.94**. Below a configurable threshold, the prediction is surfaced as low-confidence / unknown, which is what finally made honest out-of-distribution handling possible (an uploaded photo of something that isn't a tomato or mango leaf gets an honest "unknown," not a fake 0% bar).

STORY · THE BUG THAT WAS HIDING IN PLAIN SIGHT THE WHOLE TIME

Here is the quiet punchline. Go back and look at the evaluation's per-disease CSV — the *base model's* `avg_confidence` column reads `9.2e-10`, `1.2e-9`, `8.4e-10` … the same degenerate one-in-a-billion number, sitting in the shipped evaluation artifacts. The last-token-logprob confidence had been meaningless all along; it just never mattered until the demo tried to *show* it to a human. The moment you put a number in front of a person, its meaning stops being optional.

Fix robustness in the demo, not in the frozen extractor

The demo also revealed that the production disease extractor — frozen, because the evaluation numbers depend on it byte-for-byte — returns "unclassified" for perfectly valid human spellings: "dieback" (one word), "sooty mold" (US spelling), acronyms. The instinct is to fix the extractor. That instinct was correctly resisted.

Decision · A robustness wrapper in the app layer; the evaluation contract stays frozen

PROBLEM

The frozen extractor returns "unclassified" for valid variants ("dieback," "sooty mold"). Editing it would change the evaluation numbers.

DECISION

Add `app/classification.py`, which calls the production extractor *first*, then — only on an "unclassified" — applies a normalized/alias, crop-scoped fallback (dieback, sooty mould/mold, acronyms).

CONSEQUENCE

10/10 tricky cases pass in the demo; production behavior and the evaluation numbers are unchanged. What is *measured* stays frozen; what is *demoed* is more forgiving. The two are cleanly separated.

Two more touches make the demo trustworthy rather than flashy. An **adapter-verification panel** shows the LoRA rank, parameter count, and — crucially — that the adapter is actually *attached* to the base model, so a viewer can confirm they are seeing the fine-tuned model, not the base one. And memory is kept flat: the model loads once via `st.cache_resource` (one crop resident at a time), and the MLX buffer cache is bounded and cleared per run, so a long demo session doesn't creep toward an out-of-memory crash on a 24 GB machine.

LESSON · SEPARATE "MEASURED" FROM "DEMOED," AND NEVER TRUST A NUMBER YOU HAVEN'T INTERROGATED

Two lessons, tightly wound. First: the metric you report and the behavior you demo can and should have different robustness — freeze the thing you measure, forgive the thing you show, and keep them in different files. Second: a confidence, a score, any number bound for human eyes must be interrogated before it is displayed — the last-token logprob looked like a confidence and was noise.

THE EVOLUTION OF MY THINKING · THE DEMO

Before: that a demo is a thin veneer over finished machinery — wire the model to some buttons.

What changed my mind: the 1e-9 confidence. Presentation is not a veneer; it is a new interface with its own truth conditions. A number that was harmlessly meaningless in a batch job becomes a lie the instant a human reads it.

Carried forward: *the presentation layer has its own honesty burden.* Compute confidence from what the model actually said; make "unknown" a first-class answer; and prove the adapter is attached rather than asking the user to take it on faith.

EVIDENCE

Commit `088c134` ("add Streamlit inference/demo app"); `CHANGELOG.md` demo entry + `docs/DEMO_APP.md` (stream_generate confidence, threshold, unknown handling, adapter verification, bounded MLX cache); `app/inference.py`, `app/classification.py`; the degenerate base `avg_confidence` (~1e-9) visible in `reports/qwen25vl_{tomato,mango}_qlora/evaluation/per_disease.csv`.

PART XII

Making it public

The last two commits of the story are not about capability — they are about *trust*. Turning a working research repository into something a stranger can read, verify, and believe is its own discipline, and PlantDx applied to its README the same rule it applied everywhere else: a presentation must never quietly change a result.

Re-render the figures; never recompute a metric

The first README figures were small and transparent — unreadable in a dark GitHub theme. The obvious fix would be to re-run the evaluation and regenerate prettier charts. That would have been a quiet catastrophe of principle: re-running risks a different number, and a presentation pass is not allowed to move the science.

Decision · A figure script that reads results and recomputes nothing

PROBLEM
README figures were low-quality (small fonts, transparent, dark-mode-illegible). They needed re-rendering.

CONSTRAINT
A presentation pass must not touch training, evaluation, metrics, checkpoints, or datasets.

DECISION
`scripts/render_readme_figures.py` reads the *existing* `reports/<run>/evaluation/*.csv` and re-renders larger-font, 150-DPI, opaque-white (dual-theme-safe) figures. It plots exactly the evaluated values; it recomputes nothing.

CONSEQUENCE
Publication-quality figures where every plotted number still traces to a report. The README's results table is likewise read from `metrics.json`, not hand-typed — so it cannot drift from the artifacts it summarizes.

The same pass added Mermaid architecture and demo-flow diagrams, gave tomato and mango equal first-class treatment (both crops' classes, metrics, confusion matrices, per-class results), and fixed a forward-looking papercut — replacing Streamlit's deprecated `use_container_width` with the current `width="stretch"` API — so the demo would not start life emitting deprecation warnings.

LESSON · PRESENTATION IS DOWNSTREAM OF RESULTS, NEVER UPSTREAM

The direction of the arrow matters. Results produce figures; figures never produce results. A script that *reads* the reports and re-renders them enforces that arrow mechanically — you literally cannot change a number by improving a chart. This is the README-scale version of "derive, don't restate," the rule the ontology and the crop-fix both live by.

EVIDENCE

Commits `66d428e` (tag `v1.0.0`, “overhaul README as the public entry point”) and `9f08d99` (“polish repository presentation”); `scripts/render_readme_figures.py` (reads CSVs, recomputes nothing — verified in source); its commit body (“reads real data, recomputes nothing”); the `width="stretch"` change in the demo commit body.

PART XIII

Where PlantDx stands today — and what it still cannot do

A story that only tells you what works is marketing. This chapter is the honest ledger: what is real and load-bearing, what is scaffolded but unbuilt, and — most importantly — what PlantDx genuinely *cannot* do today. The value of an artifact this disciplined is that its limits are as legible as its results.

What is real, tested, and reproducible

Stage	Command	State
Dataset audit	`plantdx audit`	Real — inventory, corrupt/dup detection, deterministic checksums
Normalization	`plantdx normalize`	Real — canonical layout, SHA-256 verify, raw immutability
Ontology compiler	`plantdx ontology`	Real — DKB → typed graph, fail-closed, hash `25ae0f6d`
Vocabulary + lexicon	`plantdx vocabulary`	Real — closed vocabulary, hash `99d5833c`
Concepts / templates / corpus	`plantdx concepts | templates | generate`	Real — 1,966 captions, hash `1225eb86`
Exporters	`plantdx corpus --format …`	Real — 5 formats, byte-identical
Training	`plantdx train`	Real workflow — **tomato** + **mango** QLoRA adapters trained (Qwen2.5-VL-4bit)
Evaluation	`plantdx evaluate`	Real — two-stage, crop-agnostic, both crops measured
Demo	`streamlit run streamlit_app.py`	Real — grounded diagnosis, honest confidence, adapter proof

What is scaffolded but not built

True to the scaffold-first philosophy of Part II, several packages exist as typed stubs marking frozen future contracts — honest promises, not working code:

- The **image-grounded generation / validation / diversity** stack (the per-image diversity engine: anti-domination caps, coverage-guided sampling, MinHash near-dup) — designed in the caption framework, still stubbed. The training pipeline currently uses a simpler corpus×image cross-join, not this full engine.

- The five **per-model VLM converter bodies** (`CONVERTER_REGISTRY`) — the registry and class shapes exist and are structurally tested; the conversion bodies are not implemented.
- **QA sampling** (`qa/*`) and the human-in-the-loop review protocol — specified, unbuilt.
- `DKBLoader` — genuine dead code: a stub nothing calls, because the ontology compiler has its own loader. Flagged for a future decide-to-implement-or-retire.

What PlantDx still cannot do — stated plainly

THE HONEST LIMITS

- **Two crops, eighteen classes.** Tomato and mango leaves only. The architecture is designed to add crops additively (zero schema change), but no third crop exists.
- **One of four declared models actually trained.** Only Qwen2.5-VL has adapters and metrics. Qwen3-VL, InternVL3, and Gemma-3 are declared targets with converter scaffolding and *no* trained weights.
- **No per-image severity, ever.** This is a deliberate honesty limit, not a gap: the datasets carry no per-image severity label, so the system will not assert one.
- **The corpus is disease-level, not the full image-grounded dataset the spec describes.** The frozen `caption_framework/04` instruction-dataset builder (image cross-join with the diversity engine, per-model converters) is future work; the diversity benchmark gates are, by design, not met by the current controlled corpus.
- **No human evaluation was performed.** The QA protocol (Cohen's $\kappa \geq 0.80$, per-class review) is specified but not executed. All reported metrics are automatic.
- **Die back is near-unusable today.** 4% recall on mango die back — the model almost never emits it, because its leaf presentation is generic. Honest, and unresolved.
- **No GitHub collaboration artifacts.** Zero issues, PRs, or releases; the project's reasoning lives in commits and specs, not discussion threads.

Where it goes next

The roadmap the specs already anticipate: **M4** — image grounding + the instruction-dataset builder (cross-join the corpus with images, the diversity engine, the five per-model converters, QA); then **fine-tuning the other three declared models** on identical splits for the fair multi-model comparison the whole canonical-schema design was built to enable; then the **IEEE paper** the methodology section was drafted for. Every one of these has a place already framed in the repository — the stub is already there.

THE EVOLUTION OF MY THINKING · SHIPPING SOMETHING HONEST

Before: that "done" meant "works and looks finished."

What changed my mind: realizing that a research artifact's credibility comes as much from the clarity of its *limits* as from its results. A confident README that hides that only one of four models trained, or that die back barely works, is less trustworthy than one that says so.

Carried forward: *make the limits as legible as the wins*. The same evidence discipline that grounds a caption should ground a claim about the project itself.

EVIDENCE

`CLAUDE.md` §6 (implemented vs. stubbed inventory) and §19–20 (known issues + next milestone scope); `docs/ROADMAP.md`; `PROJECT_CONTEXT.md`; the trained-adapter reality for both crops from the two `reports/qwen25vl_*_qlora/evaluation/` trees; the die-back number from the mango `per_disease.csv`; GitHub emptiness verified via the authenticated `gh` CLI (0 issues / 0 PRs / 0 releases).

REFLECTIONS · I

Things we got wrong & things we deleted forever

A project's honesty is measured by how it talks about its mistakes. PlantDx's git history is unusually candid about its own, because so many commits are corrections of earlier ones. Here is the catalogue, drawn only from what the repository actually reversed.

Things we got wrong

What we got wrong	How the repository knows	What fixed it
Assumed the tomato data was a tidy 10-class subset	The audit found 38 classes across many crops	The normalization engine (extract only the 10)
Assumed the concept model could be a pure ontology *view*	The ontology collapses symptoms into one type, dropping the distinctions A needs	A reads the DKB, cross-linked to the ontology
Assumed grounded truth implied good grammar	The real DKB produced "a coating are visible," clause-as-noun, negation leaks	RC1's three deterministic grammar filters
Assumed the pipeline was general because it worked	Mango scored at chance through three hardcoded `"tomato"` literals	Crop derived from the dataset manifest
Assumed the last-token logprob was a confidence	It read 1e-9 on correct predictions (and sat, degenerate, in the eval CSVs the whole time)	`stream_generate` mean-emitted-token probability
Believed the code was pytest-clean without running pytest	Pytest couldn't even *collect*: a circular import + a test-module basename collision	The CI-stabilization pass (and a hard rule: run the real tool)

STORY · THE MISTAKE ABOUT MISTAKES

The most humbling entry is the last one. For several milestones the project believed it was "pytest-clean" based on ad-hoc verification scripts — and when `pytest` was finally run for real, it could not even collect the suite. A hand-written verification harness, however careful, is not the CI gate. This single embarrassment hardened into one of the project's firmest rules: *run the actual tool; do not reason about whether it would pass.*

Things we deleted forever

Deletion is a form of design. Each of these was removed on purpose, and the repository is better for the empty space:

- **`core/interfaces.py`** — a generic `Protocol` layer with zero consumers, duplicating the concrete ABCs. Deleted in the first review pass; the archetypal "speculative abstraction."
- **The `SeedDeriver` class** — collapsed into three pure functions (`image_seed`, `caption_seed`, `attempt_seed`). A class that only held functions was not earning its `self`.
- **The seven-file `dataset/converters/` package** — consolidated into one `converters.py`. Seven one-line files were file-count theater.
- **Speculative config knobs** — per-validator enable/blocking toggles, a duplicated `max_attempts`, `reproducibility.hash_algorithm`, a duplicated eval output dir. Removed as "magic settings for flexibility nobody asked for."
- **Superseded stubs, left as contracts (a nuance):** the M1 caption-ontology and vocabulary stubs were *not* deleted — they were kept as frozen future surfaces even after fresh `domain/` packages superseded them. The lesson cuts both ways: delete dead *over-scaffolding*, but keep an intentional stub that marks a real future contract.

> **LESSON · DELETION IS DESIGN, BUT DISTINGUISH DEAD CODE FROM A PROMISE**
> Every deletion above removed weight that no one was carrying on purpose. But the project also learned the opposite discipline: a typed stub that marks a genuine future milestone is not dead code — it is a contract. The skill is telling "nobody will ever call this" apart from "nobody calls this *yet*," and the tell is whether a spec points at it.

EVIDENCE

`CHANGELOG.md` "post-review simplification" and "CI stabilization pass" entries (the deletions, the circular import, the test-basename collision); `CLAUDE.md` §12 (coding philosophy) and §13.3 ("CI was not actually exercised for a long time"); the audit/normalization, concept-model, RC1, crop, and confidence corrections cited in their respective chapters above.

REFLECTIONS · II

Assumptions that changed & decisions we'd never repeat

Research assumptions that changed

ASSUMPTION → CORRECTION

- **"More caption data wins."** → *Provenance wins.* A corpus inherits the epistemics of its source; a model-written corpus can only be as trustworthy as a model that scores 7%.
- **"A capable VLM can write good-enough captions we'll filter."** → *The circularity is fatal and structural.* Post-hoc filtering catches known failures; it cannot catch the unknown ones. Generation-by-knowledge removes the failure mode instead of policing it.
- **"A cited knowledge base is the foundation."** → *Truth-in-prose invites drift; truth must be compiled.* The DKB is the source, but the ontology is the foundation downstream stages actually stand on.
- **"Grounding guarantees a good caption."** → *Grounding guarantees truth, not grammar.* Two separate machines are needed.
- **"It works, so it's general."** → *Generality is only proven by the second instance.* Mango was the audit tomato could never be.

Engineering decisions we would never repeat

Never again	Why
Trusting an ad-hoc verification script in place of the real `pytest` / `ruff` / `mypy`	It hid a suite that could not even collect. The only proof a gate passes is running the gate.
Embedding a domain value (`"tomato"`) as a literal	It made a genuinely-general pipeline silently single-instance; the second crop scored at chance.
Displaying a number before interrogating what it means	The last-token logprob "confidence" was noise (1e-9) that sat unnoticed in shipped artifacts until a human read it.
Freezing a generator on synthetic fixtures alone	Real inputs produced true-but-ungrammatical captions no fixture had. Harden against reality before freezing.
Building abstraction before a consumer exists	`interfaces.py` shipped and was deleted in the same era. Write the interface the current milestone needs.

LESSON · THE META-PATTERN UNDER ALL OF THESE

Nearly every "never again" is the same mistake in a new costume: *trusting a proxy for the real thing*. A verification script proxies for CI; a literal proxies for a derived value; a last-token logprob proxies for confidence; a fixture proxies for real data; an abstraction proxies for a need. The cure is always the same — touch the real thing: run the gate, read the manifest, measure the emitted tokens, use the real DKB, wait for the real consumer.

EVIDENCE

Synthesized from the corrections documented in Parts III–XII and their Evidence boxes; the "trust a proxy" pattern is the throughline of `CHANGELOG.md`'s correction entries (post-review simplification, CI stabilization, crop-agnostic, RC1, demo confidence) and `CLAUDE.md` §13.3.

REFLECTIONS · III

The hardest problem & the most satisfying breakthrough

The user's brief asked for "the most difficult day" and "the most satisfying breakthrough." I can give you the second honestly and must qualify the first, because the repository records *problems*, not *days*.

CANNOT BE RECONSTRUCTED

There is no log of hours, no timestamps of debugging sessions, no record of a single "most difficult day." Commit timestamps show the calendar order of milestones, not how long any problem took or how it felt to solve. So instead of inventing a day, this chapter names the hardest *problems* — the ones the evidence shows demanded the most reasoning — and the breakthrough the metrics make undeniable.

The complete index of debugging stories

Every postmortem in this document, gathered so you can see the shape of the project's troubles at a glance:

#	The bug	The tell	The root cause
1	Manifest reports 0 images	Checksum = empty-string hash	Report counts only freshly-copied files (deferred, documented)
2	"raised gall" emitted twice	Duplicate lexicon items	Same word on two axes (shape & texture)
3	`agent:alternaria_solani` label	Raw id shown as a name	Agents store name under `scientific_name`, not `canonical_label`
4	"a coating are visible"	Ungrammatical real captions	Clause-as-noun, number disagreement, negation leak (RC1)
5	Pytest can't collect	Whole suite fails to import	Circular import + test-module basename collision
6	numba `ImportError`	BERTScore tests fail	Environment's numba predates NumPy 2.x (not PlantDx's fault)
7	Mango scores at chance	Tomato ids on mango leaves	Three hardcoded `"tomato"` literals
8	Confidence = 1e-9	One-in-a-billion certainty	Last-token full-vocab logprob, not emitted-token probability

The hardest problem: the bug with no stack trace

If one problem deserves the title, it is the mango crop bug (#7) — precisely because *nothing was broken in the usual sense*. No exception, no crash, no failing assertion. The code was clean, typed, and green. A model that had genuinely learned was simply being *interpreted* through the wrong crop's lexicon, and the only symptom was a number that looked like bad luck. Diagnosing it meant refusing the easy explanation ("mango just didn't train well") and tracing a correct-looking pipeline until three innocuous strings confessed. Bugs that throw are easy; bugs that quietly return a plausible-but-wrong answer are the hard ones, and this was the purest specimen in the project.

The most *conceptually* subtle problem was different: the caption-grammar bugs (#4). There, the challenge was noticing that "true" and "grammatical" are independent properties — that a pipeline could be perfectly grounded and still unspeakable — and then fixing it *deterministically*, with filters, rather than reaching for the very kind of model the project had banned.

The most satisfying breakthrough

STORY · 7% → 94%

The breakthrough needs no qualification, because it is a measured fact. A base model that scored **7.3%** on tomato — a model that survived by calling everything healthy and hallucinating the rest — became, after QLoRA fine-tuning on a corpus *no model ever touched*, a model that scores **93.7%**. On mango, **17.5% → 82.0%**. The entire architecture — the knowledge base, the ontology, the closed vocabulary, the deterministic corpus, the leak-free splits — was a single, patient argument that grounded supervision would teach better than a model could teach itself. This is the moment the argument became a result. Everything upstream was built on faith in that number; the number arrived and kept the faith.

LESSON

The bugs that hurt most are the ones that return a wrong answer without complaining. Build the tripwires — leakage guards, golden hashes, real-data hardening — that make silent wrongness loud. And when a long architectural bet finally pays off in a metric, notice that the payoff was earned upstream, in the discipline, long before the number appeared.

EVIDENCE

Postmortems 1–8 and their sources are cited in Parts III, VI, VII, IX, X, XI and the `CHANGELOG.md` CI-stabilization entry; the 7.3%→93.7% and 17.5%→82.0% figures from `reports/qwen25vl_{tomato,mango}_qlora/evaluation/metrics.json`.

REFLECTIONS · IV

If we rebuilt it today — and if we had six more months

How we would rebuild PlantDx today

Not differently in architecture — the architecture was vindicated — but differently in *sequence*. Almost every hard lesson was learned mid-stream and then retrofitted; a rebuild would front-load them as day-one defaults:

1. **Audit before architecting, always.** The 38-vs-10 surprise would be the first commit's output, not a mid-project discovery.
2. **Wire the real CI gates before writing logic.** Given that "pytest couldn't collect" went unnoticed for milestones, a rebuild runs `ruff` / `mypy` / `pytest` from commit one and never trusts a bespoke script.
3. **Derive every domain value from data, from the start.** No `"tomato"` literal ever exists; crop is read from the manifest on day one, so "add a crop" is free from the beginning rather than earned by a bug.
4. **Compute display numbers correctly the first time.** Confidence = mean emitted-token probability from the outset; the 1e-9 placeholder never ships, not even into a batch artifact.
5. **Harden the generator on the real DKB before any freeze.** Make "run against reality" a gate, not a late audit — the three grammar bugs would be caught before v0.6, not in RC1.
6. **Keep determinism and content hashes from the first artifact.** They cost little early and become priceless — the golden-hash tripwire is only as good as how early it exists.

> **STORY · THE SAME HOUSE, BUILT IN A BETTER ORDER**
>
> Every item above is a lesson the project already learned — just later than it wished. A rebuild would not change what PlantDx *is*; it would change *when* each discipline arrived, moving them from "retrofit after the bug" to "default before the code." The architecture needs no apology. The schedule does.

Open research problems

- **Observationally-ambiguous classes.** Die back at 4% recall is the open wound: how should a knowledge-grounded system handle a class whose leaf appearance is genuinely generic? The DKB *predicted* the difficulty; it did not solve it.
- **Image-grounded diversity.** The per-image diversity engine (anti-domination, coverage-guided sampling, MinHash) is specified but unbuilt; the current corpus is disease-level and does not meet the diversity gates by design. Does the full engine change downstream accuracy, or only linguistic variety?

- **The fair multi-model comparison.** The entire canonical-schema + converter design exists to fine-tune four models on *identical* content and splits — but only one is trained. The comparison the project was architected to make has not yet been made.
- **Per-image severity without a severity label.** Severity honesty forbids asserting a stage today; unlocking it cleanly would require a new, separate per-image annotation task — an open data problem, not just a code one.
- **Curation cost vs. crop scale.** Adding a crop is architecturally free but *epistemically expensive* — it needs an expert-curated, cited DKB entry. How far does that scale before curation, not code, is the bottleneck?

If we had six more months

THE SIX-MONTH PLAN THE SPECS ALREADY IMPLY

1. **Months 1–2 — M4 image grounding.** Build the instruction-dataset builder and the diversity engine; implement the five per-model converter bodies; run the QA protocol ($\kappa \geq 0.80$) that is specified but never executed.
2. **Months 3–4 — the multi-model comparison.** Fine-tune Qwen3-VL, InternVL3, and Gemma-3 on the identical splits and finally publish the four-way zero-shot-vs-fine-tuned table the architecture was built for.
3. **Month 5 — attack die back and add a third crop.** Prove crop-additivity in practice, not just in principle; investigate the observationally-ambiguous-class problem head-on.
4. **Month 6 — the paper.** Write the IEEE manuscript the methodology section (`caption_framework/07`) and the ontology contribution (`ontology_design/13`) were already drafted for.

EVIDENCE

The rebuild lessons synthesize the corrections of Parts III–XII. The roadmap and open problems from `CLAUDE.md` §7 & §20 (M4 scope, the diversity engine, the four-model comparison), `docs/ROADMAP.md`, the drafted paper sections `caption_framework/07_ieee_methodology_section.md` and `ontology_design/13_paper_contribution.md`, and the QA protocol in `caption_framework/05_quality_assurance_protocol.md` (specified, not executed).

CODA

The evolution of the engineer

The brief for this document asked for something unusual and exactly right: don't describe how the *software* evolved — describe how the *engineer* did. Read the git history not as a changelog but as a bildungsroman, and a person's mind comes into focus through the corrections they chose to make. Here is that arc, distilled — each stage a belief that a later commit revised.

THE ARC, MILESTONE BY MILESTONE

Stage	What I believed before	The principle I left with
The idea	More data wins.	**Provenance beats volume.**
The scaffold	Seriousness = abstraction.	**Simple, explicit, less clever.**
The data	Datasets are as described.	**Audit before you architect.**
The knowledge base	A KB is a lookup table.	**Centralize truth, and cite it.**
The ontology	Cited prose is a foundation.	**Compile prose into structure once.**
The vocabulary	Grammar will keep captions honest.	**Close the vocabulary; verify against reality.**
The captions	Correct facts imply good sentences.	**Truth ≠ grammar; separate the two.**
Training	The model is the main event.	**The model is the easy part; guard the data boundaries.**
Evaluation	Metrics are a formality.	**An evaluation you can't trust is worse than none.**
Mango	Works ⇒ general.	**Derive domain values; don't embed them.**
The demo	Presentation is a veneer.	**The presentation layer has its own honesty burden.**
Going public	Done = looks finished.	**Make the limits as legible as the wins.**

Stack those principles and a single sensibility emerges, the one this whole project is really a portrait of: **a refusal to trust a proxy for the real thing.** Not the model's guess in place of a cited fact. Not a literal in place of a derived value. Not a fixture in place of the real data. Not a verification script in place of the actual gate. Not a plausible number in place of an interrogated one. Not an abstraction in place of a need. Every milestone taught the same lesson in a new dialect, and the engineer who finished PlantDx is someone who now reaches, by reflex, for the real thing first.

STORY · WHAT THE READER SHOULD BE ABLE TO DO NOW

The goal of this document was that, having read it, you could explain PlantDx to a professor, a researcher, an ML engineer, or an interviewer *without opening the repository* — and, more than that, defend *why every part of it exists*. Why a knowledge base and not an LLM. Why an ontology and not raw prose. Why a closed vocabulary. Why deterministic captions. Why QLoRA on a laptop. Why two-stage evaluation. Why crop is read from a manifest. Why confidence comes from emitted tokens. If those "whys" now feel less like facts to memorize and more like consequences of a single stubborn idea — that a model too weak to diagnose must never be trusted to describe — then the story did its job.

PlantDx did not begin as an architecture. It began as a doubt about a model's honesty — and it grew, one correction at a time, into a system whose honesty you can check by hand.

APPENDIX

Evidence index & provenance

Every material claim in this story is intended to trace to one of the artifacts below. This index is the map back to the source.

A · Timeline of record (commits & tags)

Date	Commit	Tag	What it added to the story
2026-07-11	`12efddd`	v0.1-scaffold	The repository shell; the first “cut the abstraction” review
2026-07-11	`3e73d0f`	v0.3-normalization	Audit + normalization; the 38-vs-10 surprise
2026-07-12	`2125fc9`	v0.4-ontology	The typed knowledge graph; hash `25ae0f6d`
2026-07-12	`584f49e`	v0.5-stabilization	CI truth: pytest couldn’t even collect
2026-07-12	`7b45bb2`	v0.5-vocabulary	Closed vocabulary; cross-axis & agent-name bugs
2026-07-13	`05a98e1`	v0.6-generation / -corpus-freeze	Concepts, templates, corpus; RC1 grammar fixes; hash `1225eb86`
2026-07-13	`8ad2726`	v0.8-tomato-training	The QLoRA training workflow
2026-07-14	`2d11adf`	v1.0-evaluation	Two-stage evaluation; 7%→94%; the numba bug
2026-07-14	`380b18e`	—	Mango training pipeline
2026-07-14	`673ed77`	v0.2.0	Crop-agnostic evaluation; the three `"tomato"` literals
2026-07-17	`088c134`	—	The Streamlit demo; the 1e-9 confidence fix
2026-07-17	`66d428e` / `9f08d99`	v1.0.0	Public README + re-rendered figures

B · Primary artifacts

Artifact	What it grounds
`knowledge_base/dkb.json`	18 diseases × 46 fields; 21 cited sources; 4 documented disagreements; target_models; observability & pes pathogen notes
`caption_framework/00,07 + README`	The seven invariants; the no-model argument; severity honesty; diversity strategy
`ontology_design/01`	Why the ontology exists; the A/B/ chosen trade-off; th interoperability stance
`reports/qwen25vl_{tomato,mango}_qlora/evaluation/`	`metrics.json` + `per_disease.cs` — every base and fine-tuned number including the degenerate base confidence and die back recall
`docs/{ONTOLOGY,VOCABULARY,CONCEPTS,CORPUS,TRAINING,EVALUATION,DEMO_APP,KNOWN_ISSUES}.md`	Per-stage usage + the deferred-bug postmortems
`CHANGELOG.md`, `docs/adr/000{1,2}`	The chronological “why” spine and tw accepted architecture decisions
Golden-hash tests under `tests/unit/*/test_*_determinism.py`	The four content hashes as covenant

C · Provenance of this document

Reconstructed from the repository state at commit `9f08d99` on `main`. Sources: the full git history (16 commits, 12 tags), the authenticated GitHub CLI (**0 issues / 0 PRs / 0 releases** — so no discussion threads exist to draw on), the in-repository specifications and developer docs, the generated evaluation reports, and the configuration files. Where evidence was thin or absent — the author's personal motivation, the inspiring literature, any CNN/CLIP baseline, any cloud/A100 or Hugging Face `Trainer` attempt, Molmo/Pixtral/Florence, a literal "most difficult day," executed human QA — this document says so explicitly rather than inventing. Every material claim is intended to be traceable to an artifact named in its Evidence box. Nothing here was assumed; accuracy was preferred over completeness at every turn.

The Story of Building PlantDx — end of document. Reconstructed only from repository evidence; gaps marked, not filled.